

Network Computing

(058172)



POLITECNICO
MILANO 1863

Datacenter Layer 3 Load Balancing

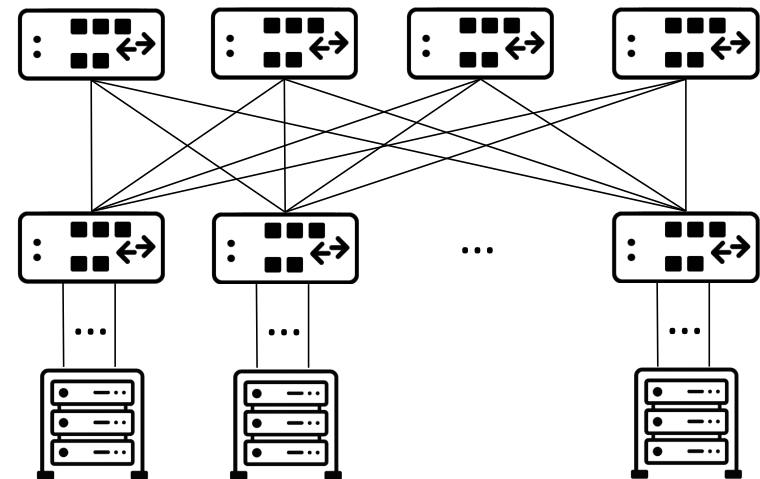
Some parts of the slides are inspired from Al-Fares' talk at NSDI 2010, Katta's talk at SOSR and Mohammed Alizadeh lectures on datacenter networks

Content

- What is Layer 3 Load Balancing and why we need it
- We will look at three specific solutions:
 - ECMP: a standard solution adopted in production networks
 - Hedera: an SDN/OpenFlow based solution proposed by the research community
 - HULA: a P4 based solution proposed by the research community

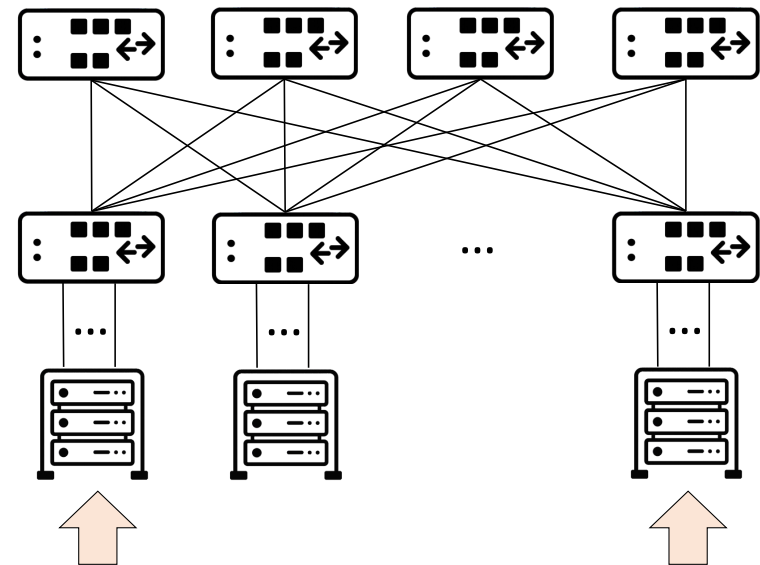
Recap: datacenters

- Many distributed applications run in a datacenter (e.g., big data, HPC, web services, ML serving)



Recap: datacenters

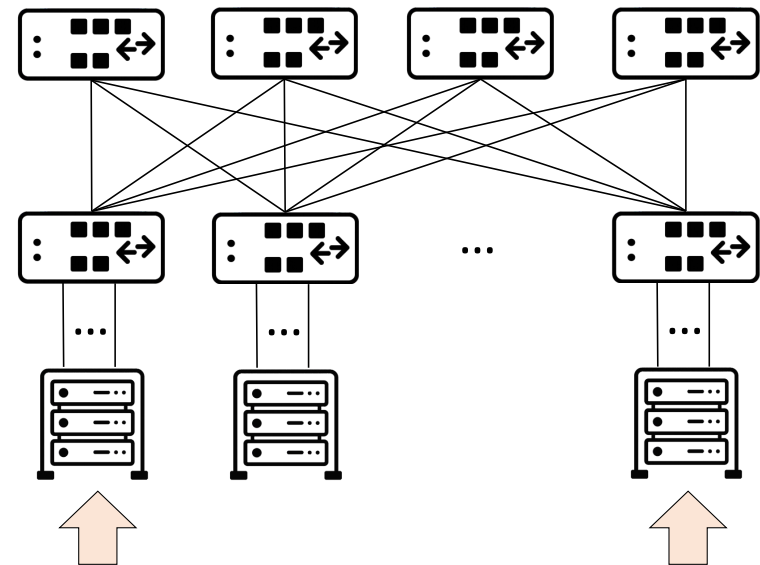
- Many distributed applications run in a datacenter (e.g., big data, HPC, web services, ML serving)
- Those generate traffic *within* the datacenter with very variable characteristics



Recap: datacenters

Cloud workloads

- Applications are split in microservices
- Databases and caches are distributed
- User requests generate many internal service calls



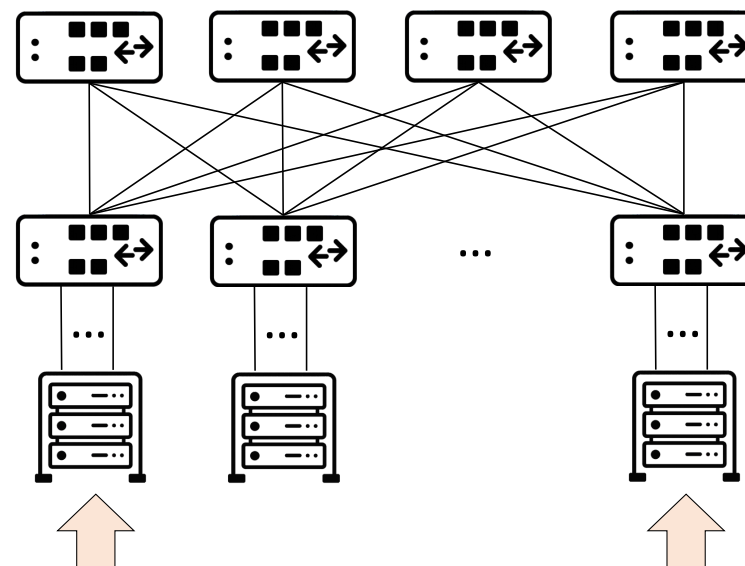
Recap: datacenters

Cloud workloads

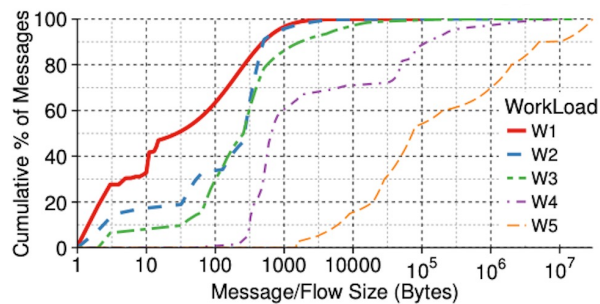
- Applications are split in microservices
- Databases and caches are distributed
- User requests generate many internal service calls

AI workloads

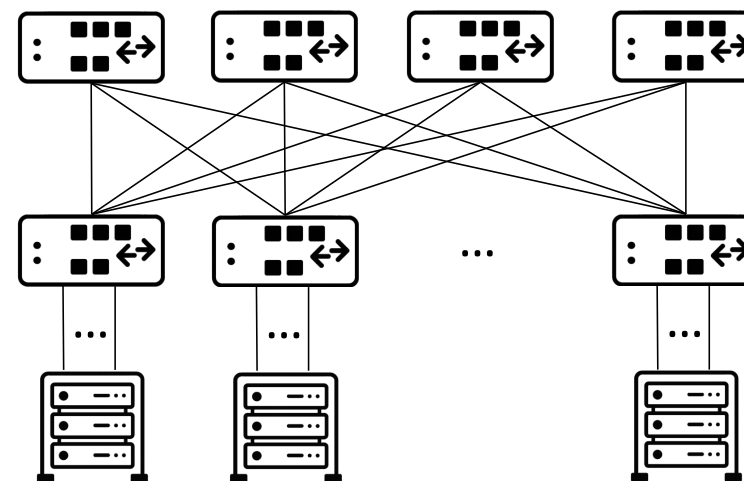
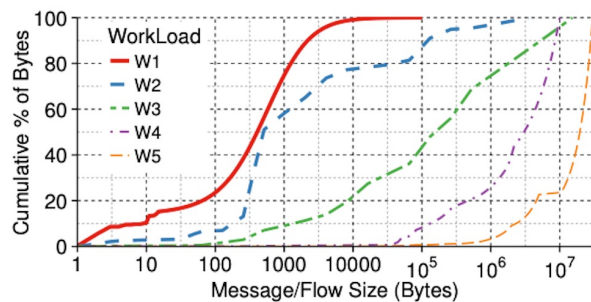
- Models are too large for a single machine
- Training uses many GPUs across servers
- Communication happens every training iteration



Recap: datacenters



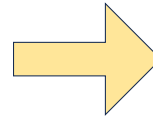
- W1 Accesses to a collection of memcached servers at Facebook, as approximated by the statistical model of the ETC workload in Section 5 and Table 5 of [5].
- W2 Search application at Google [29].
- W3 Aggregated workload from all applications running in a Google datacenter [29].
- W4 Hadoop cluster at Facebook [27].
- W5 Web search workload used for DCTCP [2].



Recap: datacenters



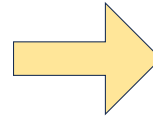
Short messages
(e.g., query, coordination)



low latency



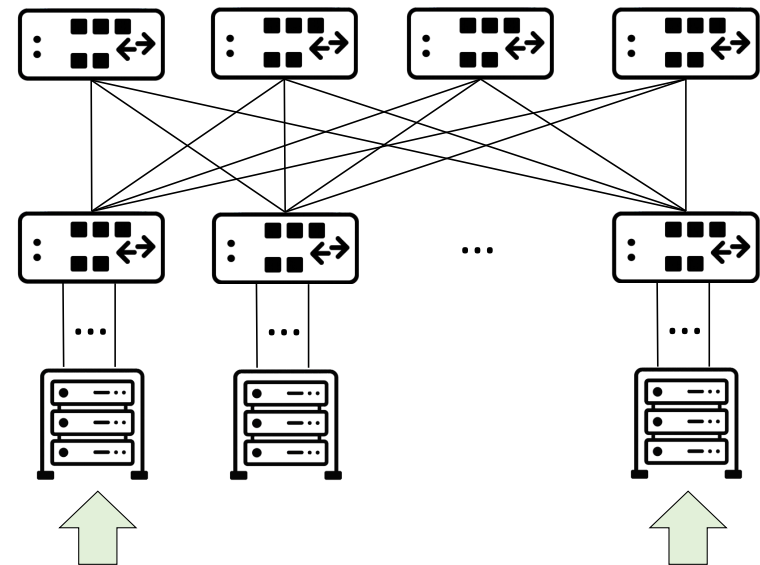
Large flows
(e.g., data update, backup)



high throughput

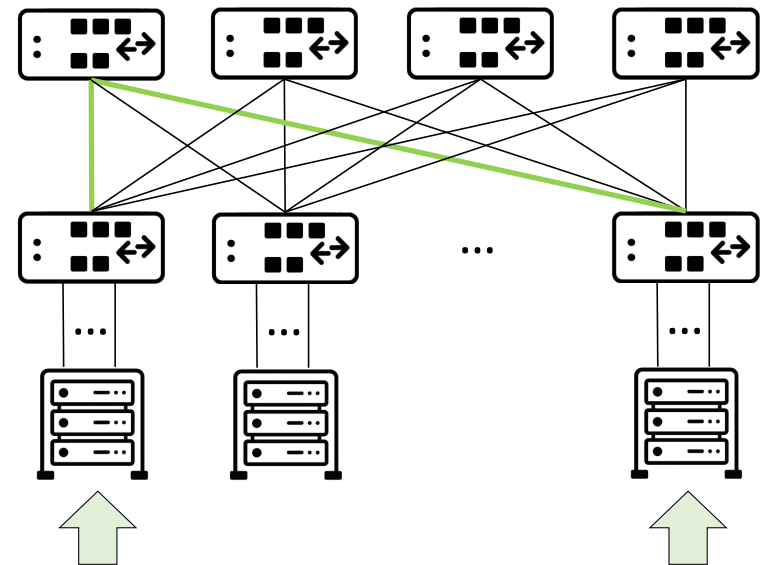
Recap: datacenters

- To best support distributed applications we need a high bisection bandwidth.
- Lots of paths available from one rack to the other



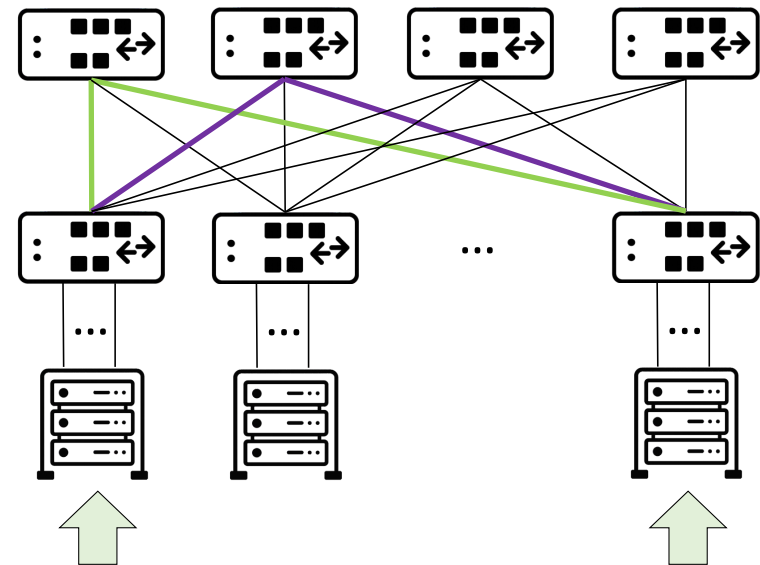
Recap: datacenters

- To best support distributed applications we need a high bisection bandwidth.
- Lots of paths available from one rack to the other



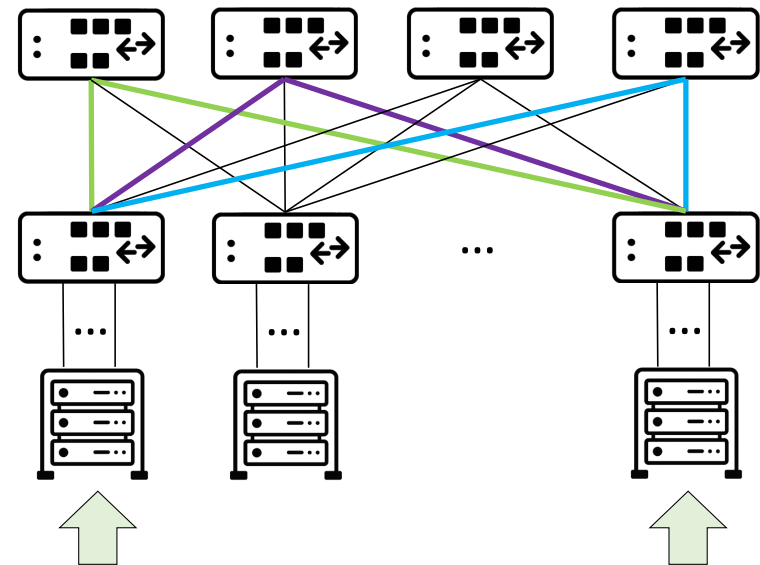
Recap: datacenters

- To best support distributed applications we need a high bisection bandwidth.
- Lots of paths available from one rack to the other



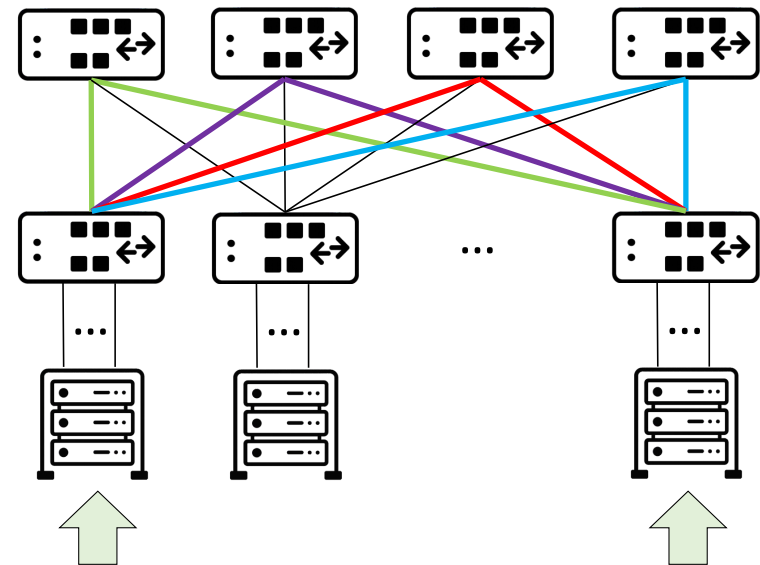
Recap: datacenters

- To best support distributed applications we need a high bisection bandwidth.
- Lots of paths available from one rack to the other



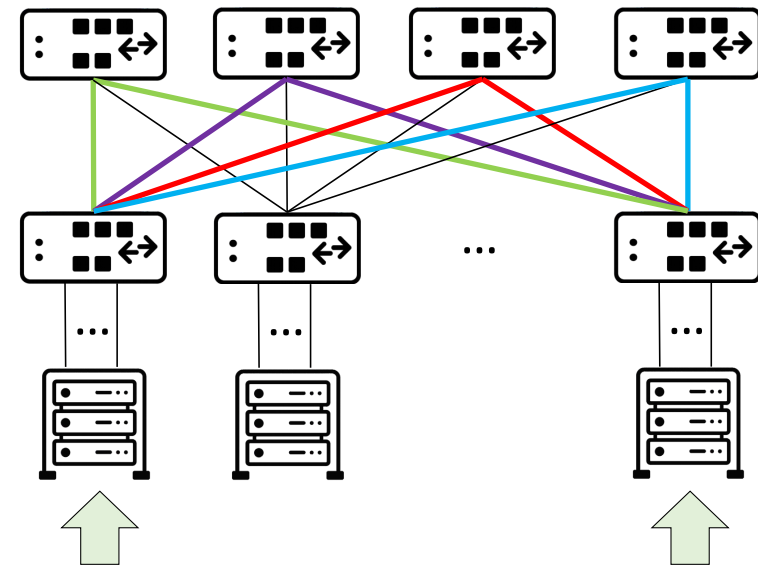
Recap: datacenters

- To best support distributed applications we need a high bisection bandwidth.
- Lots of paths available from one rack to the other



Recap: datacenters

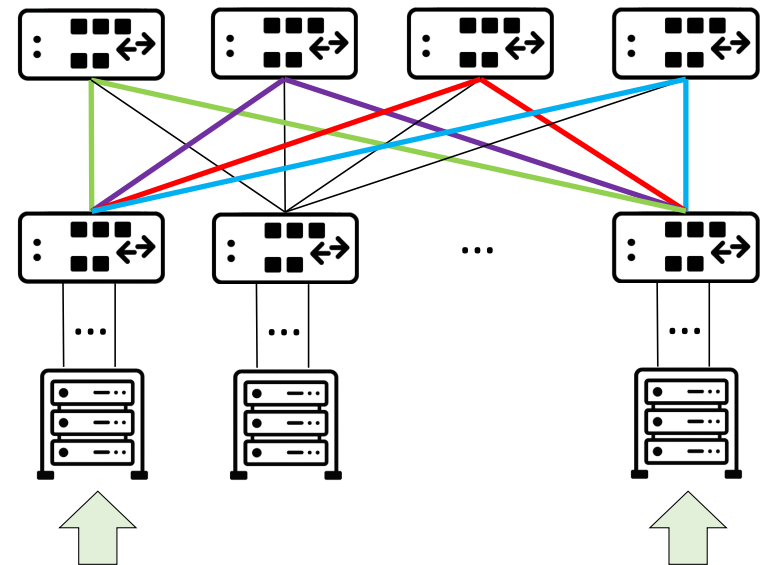
- To best support distributed applications we need a high bisection bandwidth.
- Lots of paths available from one rack to the other
- How to best utilize network resources?



Recap: datacenters

- To best support distributed applications we need a high bisection bandwidth.
- Lots of paths available from one rack to the other
- How to best utilize network resources?

This is what L3 load balancing does!



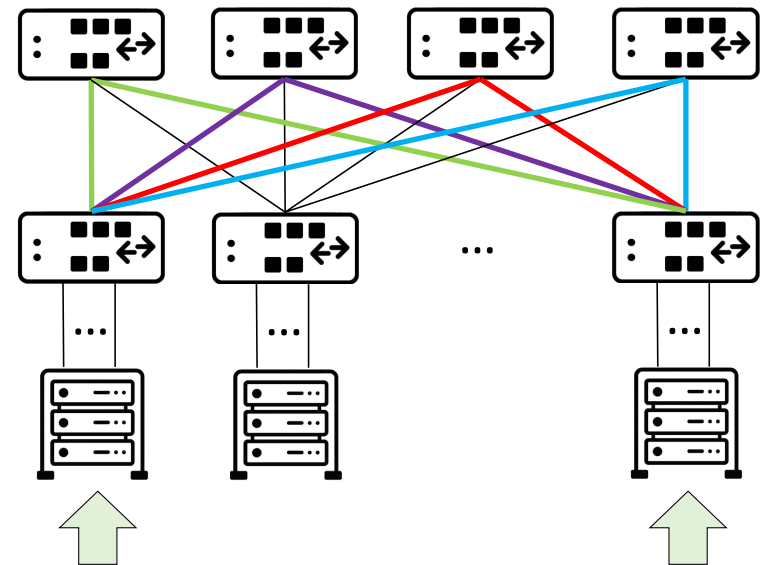
Recap: datacenters

- To best support distributed applications we need a high bisection bandwidth.
- Lots of paths available from one rack to the other
- How to best utilize network resources?

This is what L3 load balancing does!

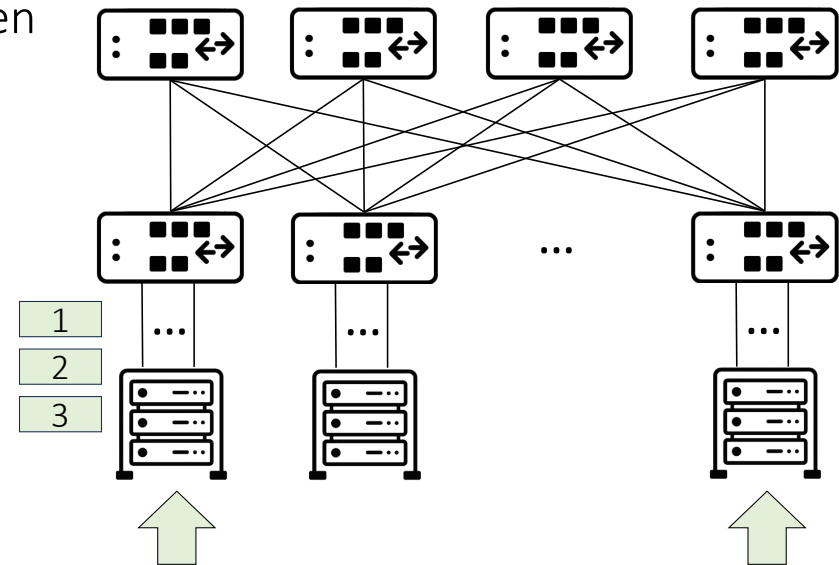
This is hard because:

- Flows come and go
- Mix of short and long flows



A thought example

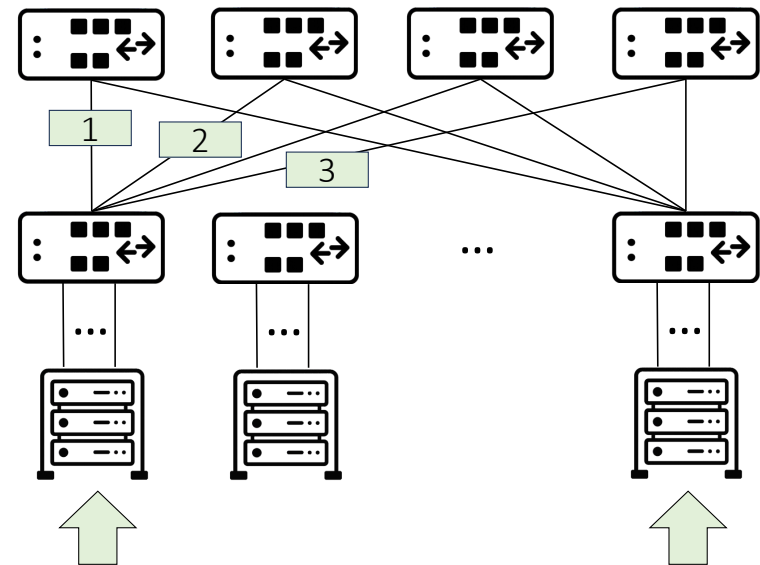
- Let's assume an application requires the two green racks to communicate
- Specifically, the rack on the left has to send three packets (P1, P2 and P3) to the rack on the right
- How to optimally route them?



Packet spraying

- You might want to *spray* the packets to all the possible paths

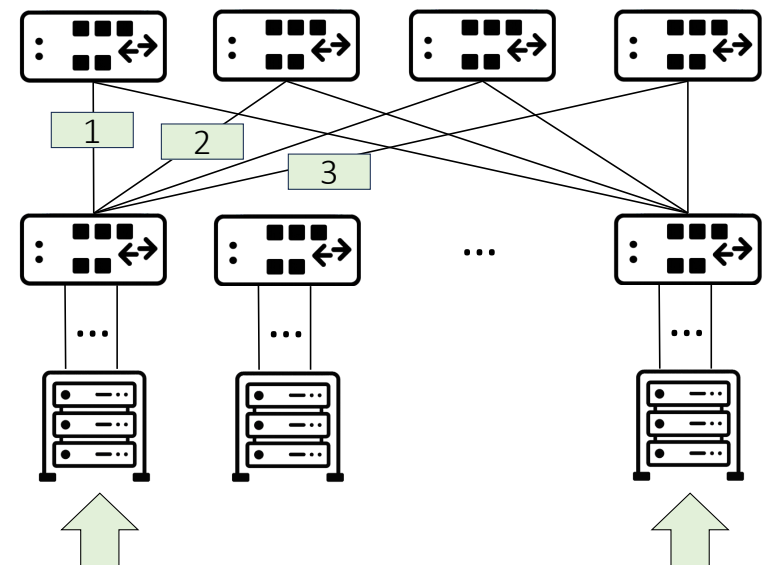
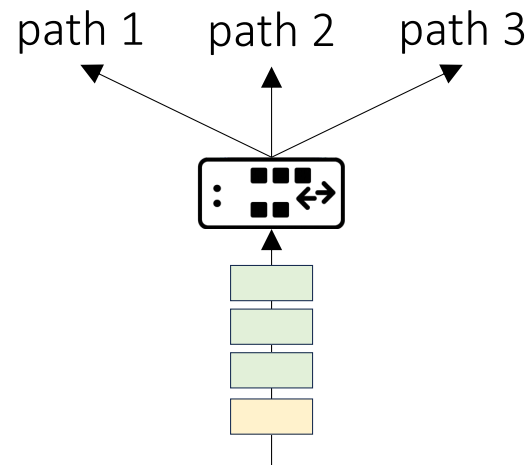
Good traffic is well spread (even with heavy flows)



Packet spraying

- You might want to *spray* the packets to all the possible paths

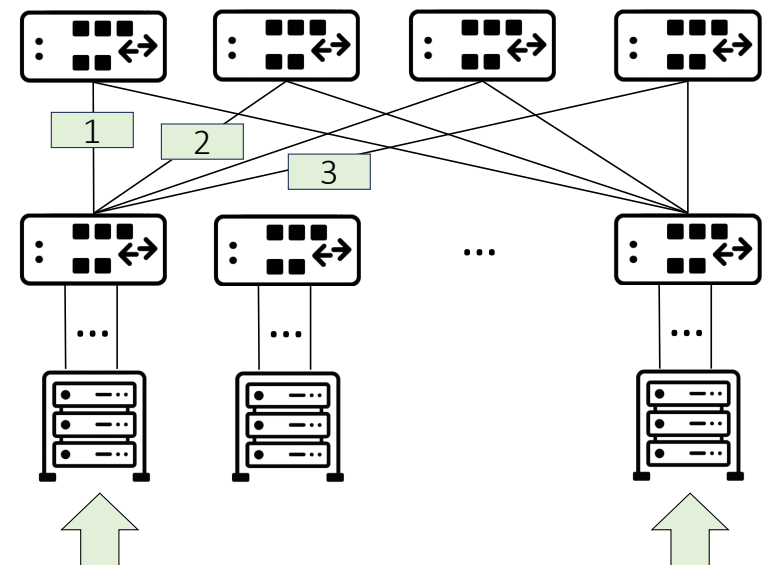
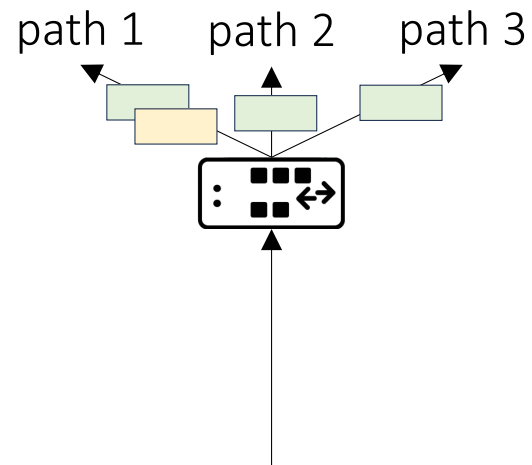
Good traffic is well spread (even with heavy flows)



Packet spraying

- You might want to *spray* the packets to all the possible paths

Good traffic is well spread (even with heavy flows)

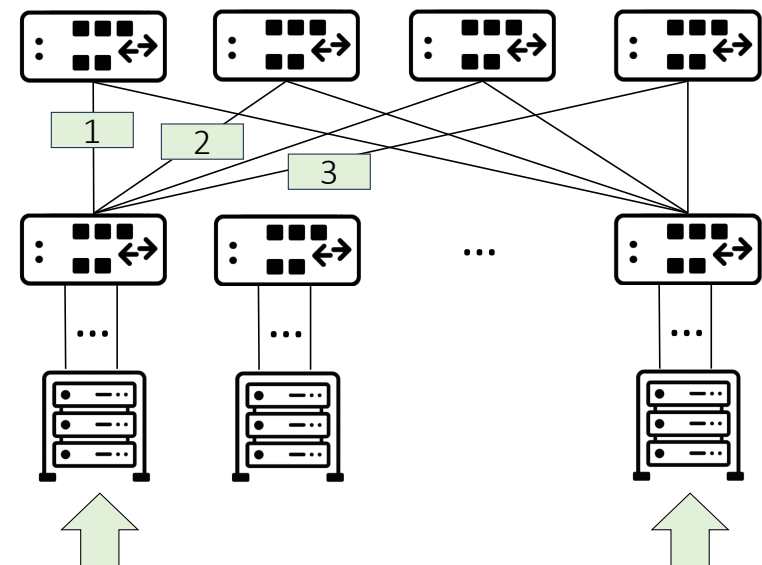
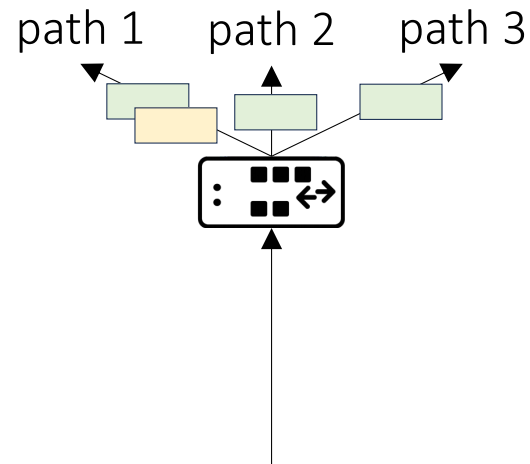


Packet spraying

- You might want to *spray* the packets to all the possible paths

Good traffic is well spread (even with heavy flows)

Bad ?

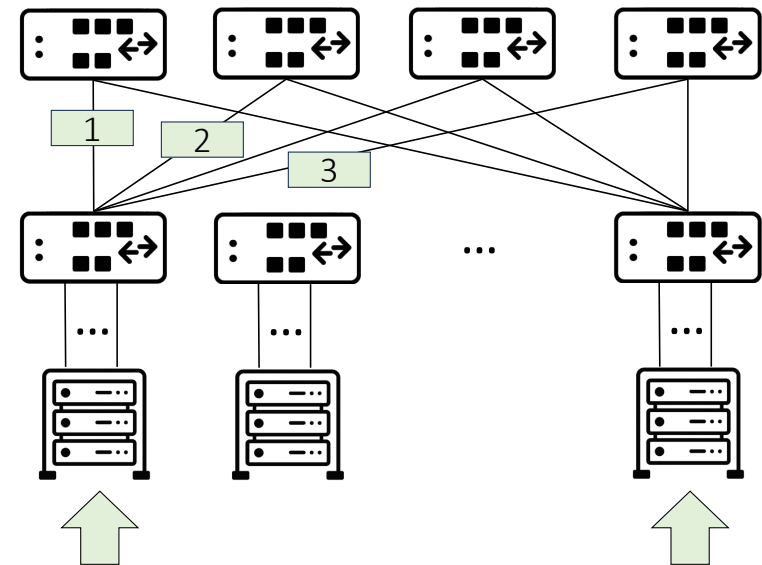


Packet spraying

- You might want to *spray* the packets to all the possible paths

Good traffic is well spread (even with heavy flows)

Bad it makes life hard to congestion control



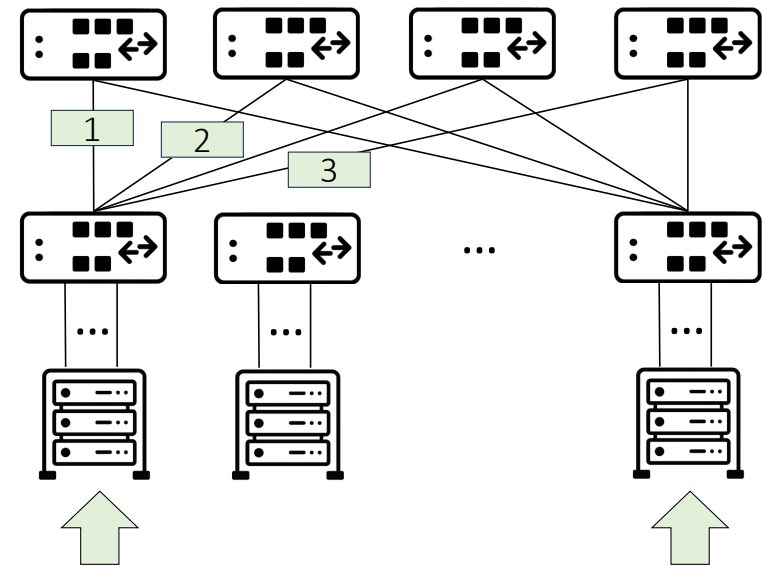
Packet spraying

- You might want to *spray* the packets to all the possible paths

Good traffic is well spread (even with heavy flows)

Bad it makes life hard to congestion control

Why?



Packet spraying

- You might want to *spray* the packets to all the possible paths

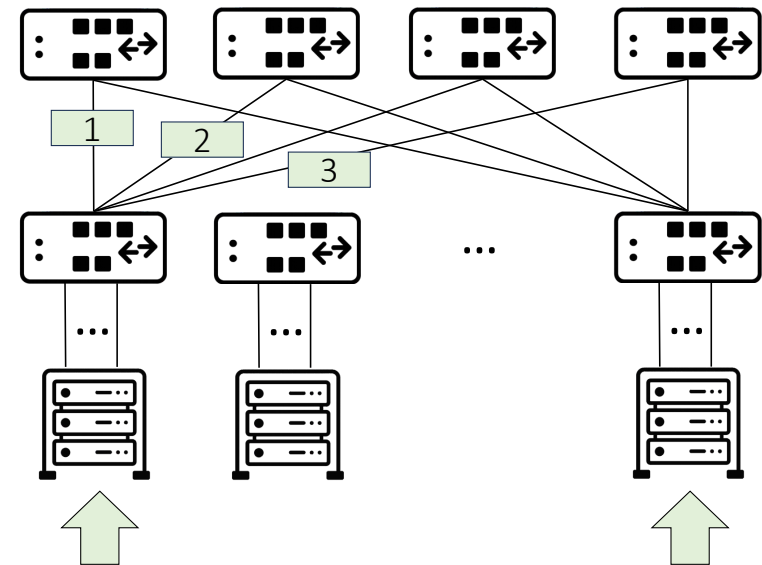
Good traffic is well spread (even with heavy flows)

Bad it makes life hard to congestion control

Why?

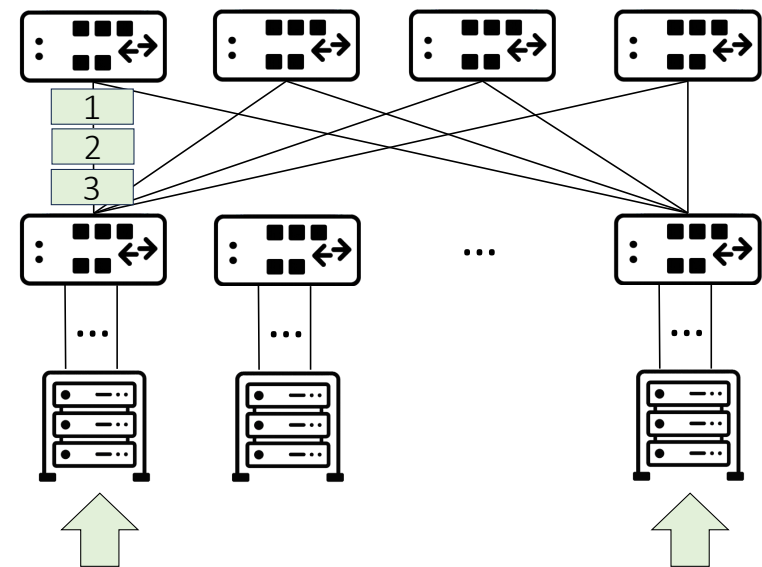
Let's take TCP as example:

- Packet reordering can trigger 3 or more duplicated ACKs (causing the sender to fast retransmit)
- Sender will reduce the sending rate



Equal Cost Multi Path (ECMP)

- A solution is to use per-flow load balancing (i.e., all the packets belonging to a flow pick the same path)

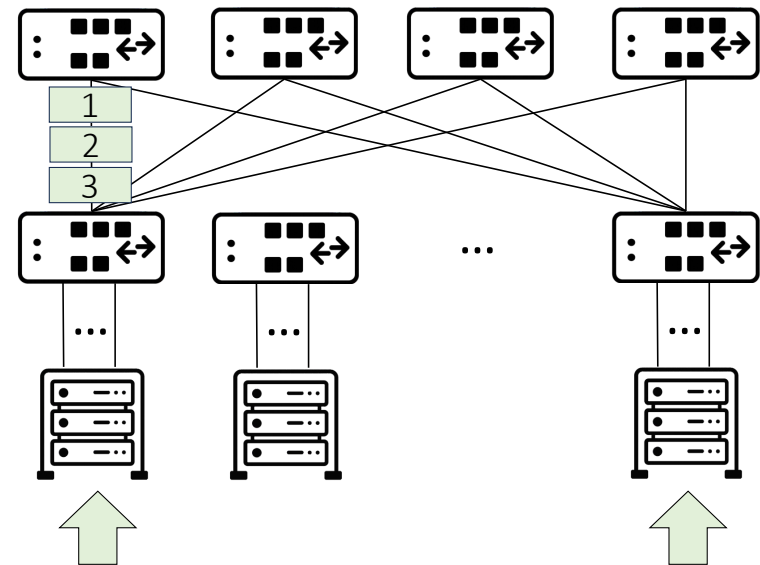


Equal Cost Multi Path (ECMP)

- A solution is to use per-flow load balancing (i.e., all the packets belonging to a flow pick the same path)

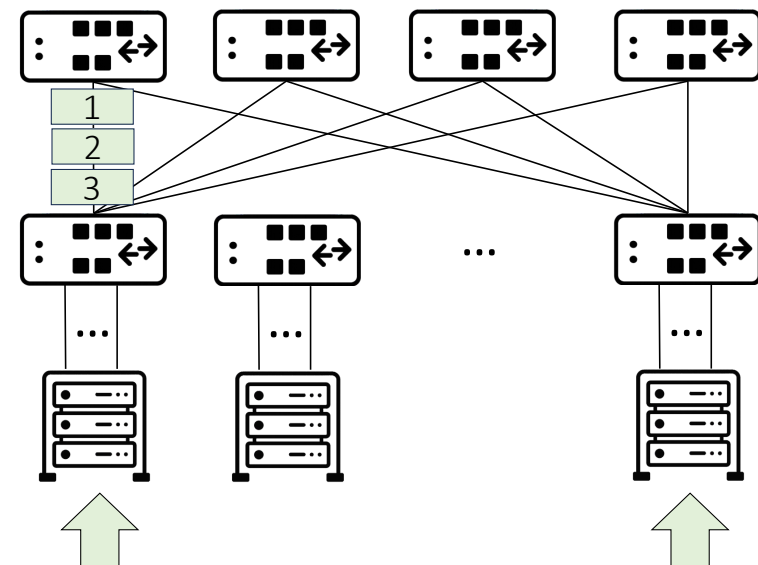
Idea (part 1) the routing protocol discovers k equal-cost paths to a destination

Idea (part 2) spread flows over these paths



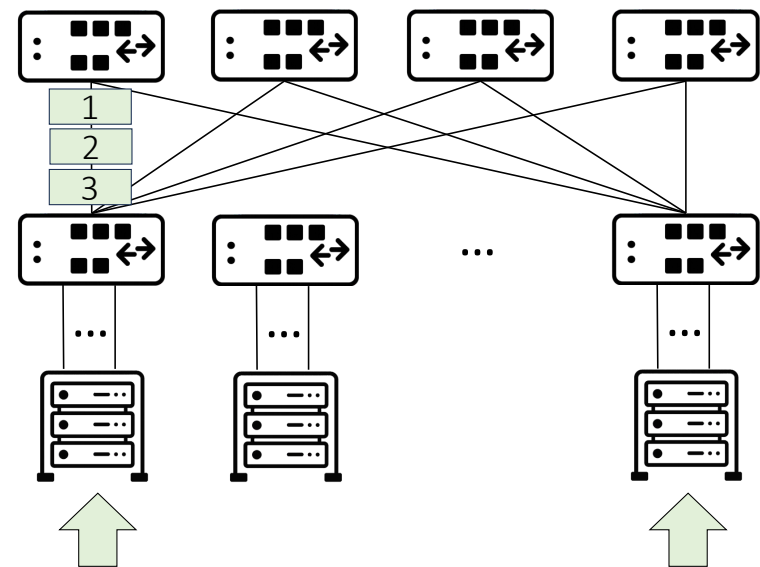
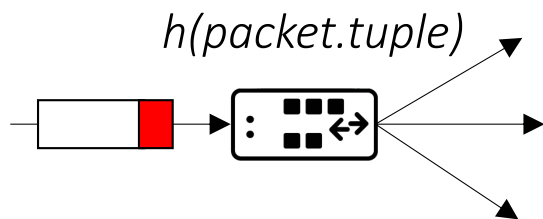
Equal Cost Multi Path (ECMP)

- This can be implemented with a hash function
 - `port.output = hash(packet.tuple)`
 - The tuple can be sourceIP, destination IP, L4 ports (this is switch implementation dependent)



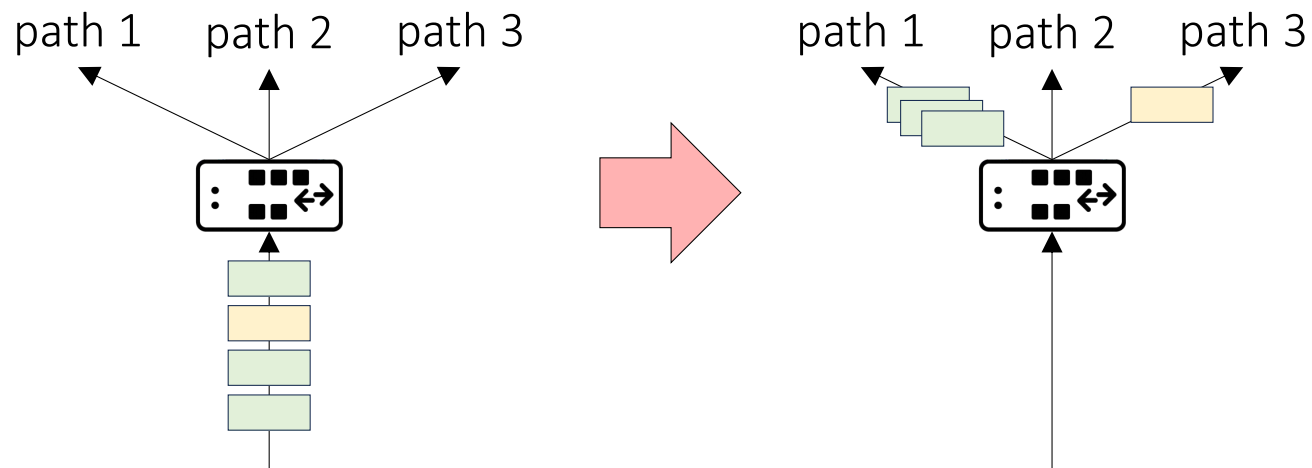
Equal Cost Multi Path (ECMP)

- This can be implemented with a hash function
 - `port.output = hash(packet.tuple)`
 - The tuple can be sourceIP, destination IP, L4 ports (this is switch implementation dependent)



1. Parse IP destination
2. Check table with all equal-costs output ports for that destination
3. Pick one randomly (using a hash)

Equal Cost Multi Path (ECMP)



ECMP: pick among equal-cost paths by hashing some fields of the packet header

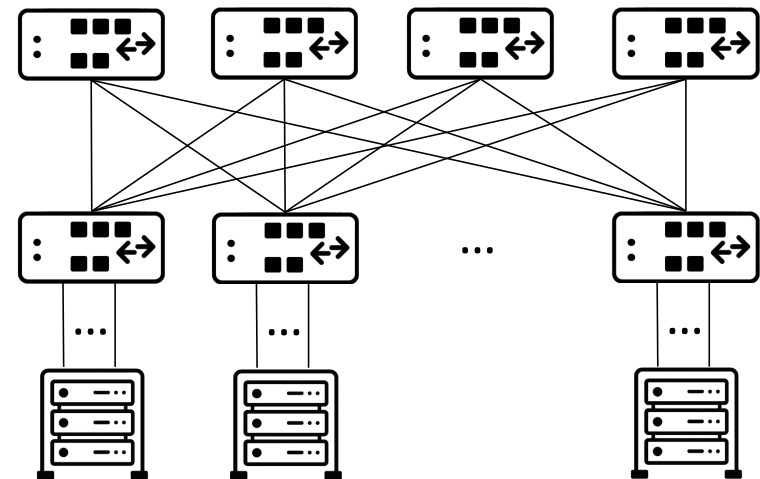
Equal Cost Multi Path (ECMP)

- ECMP: pick among equal-cost paths by hashing some fields of the packet header

Good

- a flow follows a single path (no packet reordering)

Bad ?



Equal Cost Multi Path (ECMP)

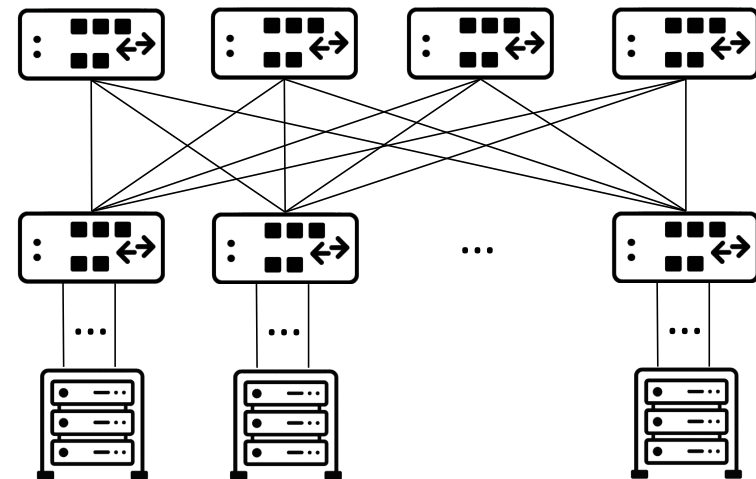
- ECMP: pick among equal-cost paths by hashing some fields of the packet header

Good

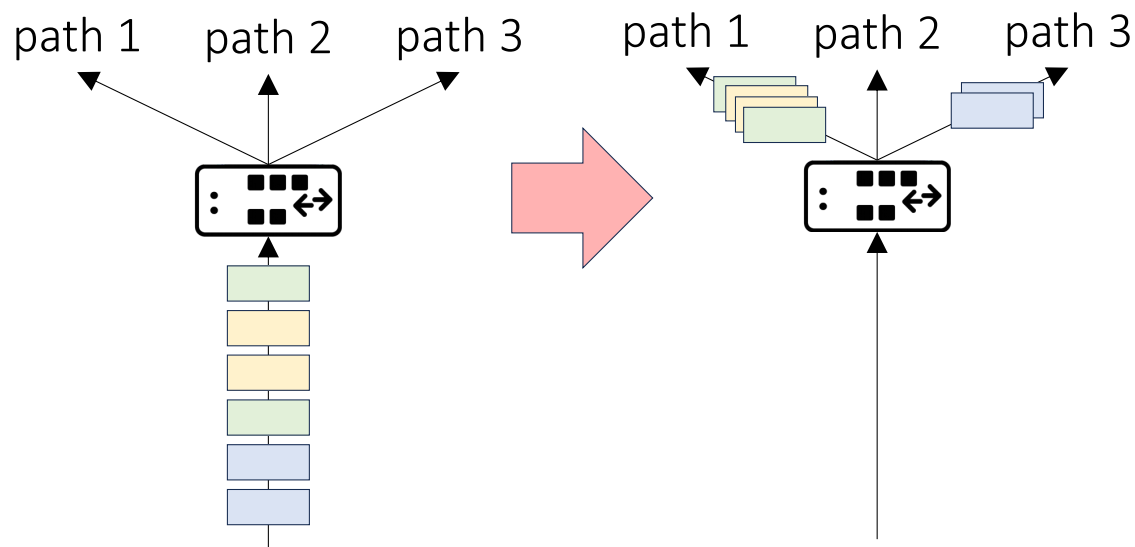
- a flow follows a single path (no packet reordering)

Bad

- The performance depend on traffic characteristics (hash collisions)
- Non optimal load balancing

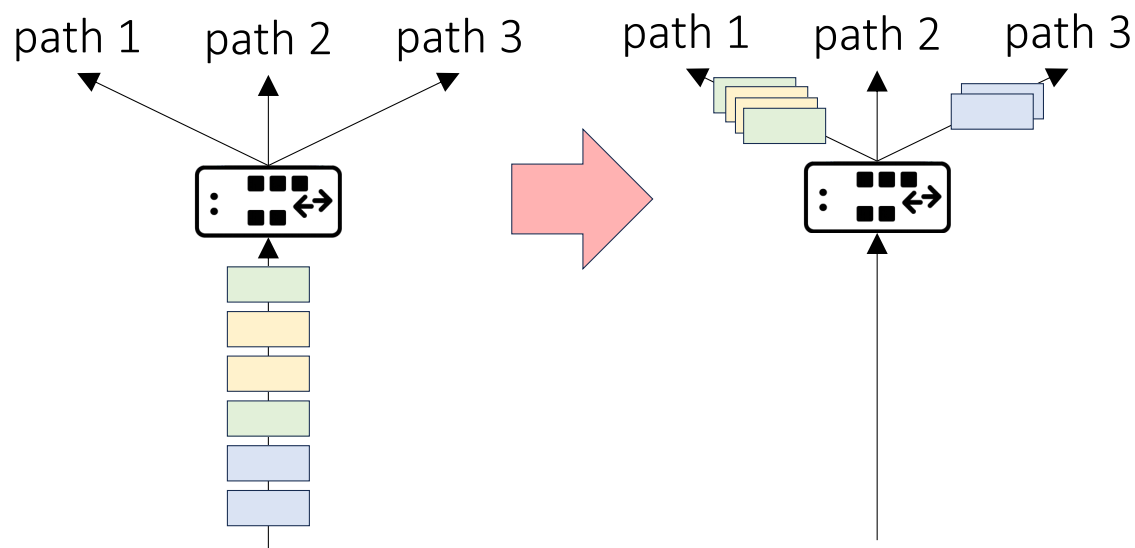


A thought example on ECMP



- Let's assume that we have 3 flows (green, yellow and blue)
- ECMP sends green and yellow out to the same port, while blue on a different one
- What's the network capacity experienced by the flows?

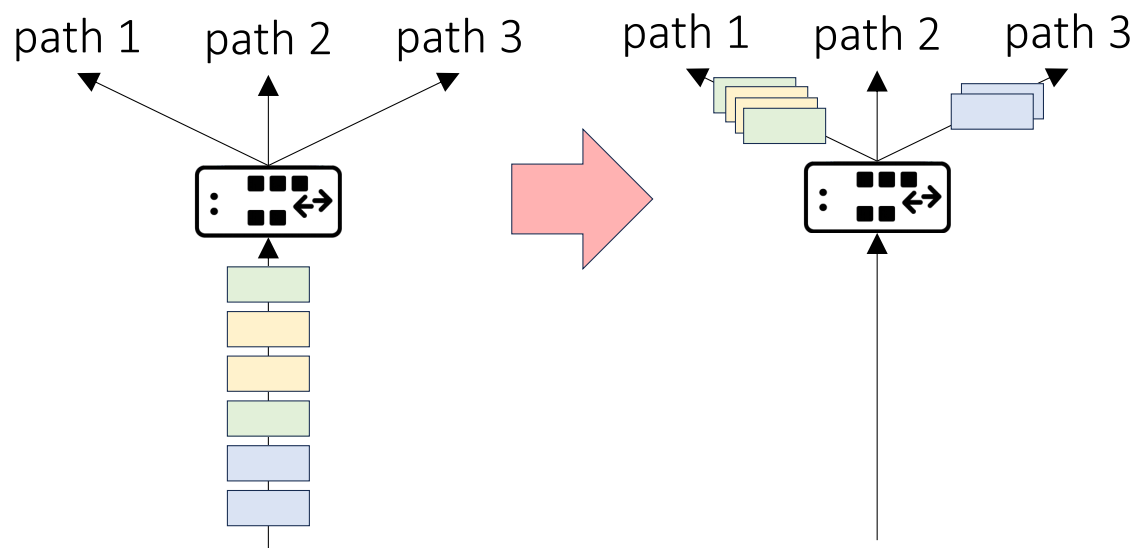
A thought example on ECMP



- Green and yellow will share the link capacity (50% each)
- Blue will get 100%
- One link will be not used (wasted bandwidth)

- Let's assume that we have 3 flows (green, yellow and blue)
- ECMP sends green and yellow out to the same port, while blue on a different one
- What's the network capacity experienced by the flows?

A thought example on ECMP



- Green and yellow will share the link capacity (50% each)
- Blue will get 100%
- One link will be not used (wasted bandwidth)

Does this remind you something we have seen?

- Let's assume that we have 3 flows (green, yellow and blue)
- ECMP sends green and yellow out to the same port, while blue on a different one
- What's the network capacity experienced by the flows?

ECMP in production

2015

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U.C. Santa Barbara ³Princeton University

ABSTRACT

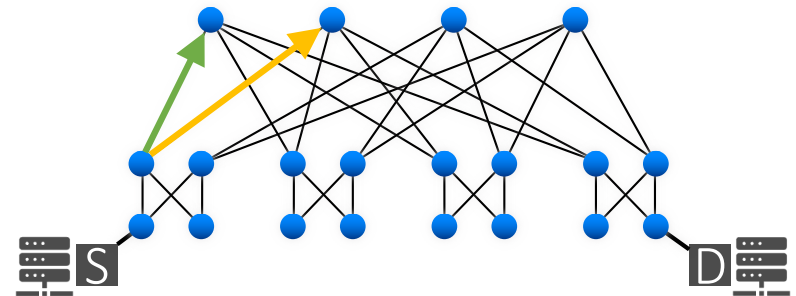
Debugging faults in complex networks often requires capturing and analyzing traffic at the packet level. In this task, datacenter networks (DCNs) present unique challenges with their scale, traffic volume, and diversity of faults. To troubleshoot faults in a timely manner, DCN administrators must *a)* identify affected packets inside large volume of traffic; *b)* track them across multiple network components; *c)* analyze traffic traces for fault patterns; and *d)* test or confirm potential causes. To our knowledge, no tool today can achieve both the specificity and scale required for this task.

We present Everflow, a packet-level network telemetry system for large DCNs. Everflow traces specific packets by implementing a powerful packet filter on top of “match and mirror” functionality of commodity switches. It shuffles captured packets to multiple analysis servers using load balancers built on switch ASICs, and it sends “guided probes” to test or confirm potential faults. We present experiments

proactive model of DCN management, where infrastructure observes, analyzes, and corrects faults in near-real time.

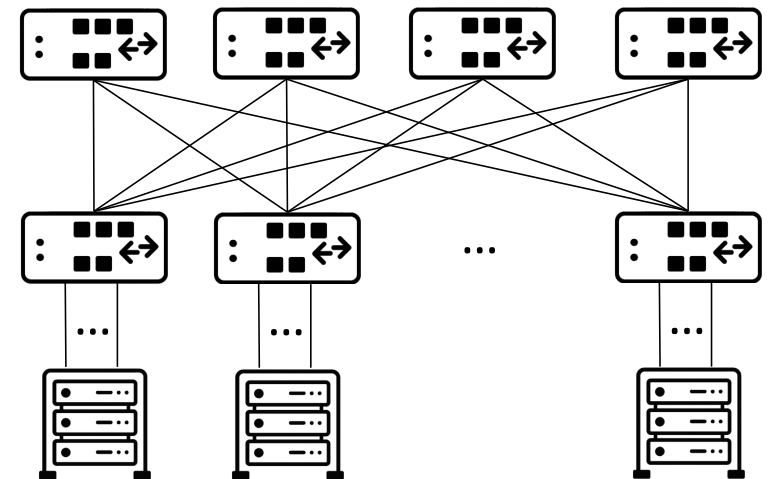
Understanding and debugging faults in DCNs is challenging because faults come in all shapes and sizes. For example:

1. Some packets may experience abnormally high delay between servers *A* and *B*, but it may not be clear which of the many links is responsible.
2. Packets destined for a specific set of servers may be dropped, even when the packet-drop counters at switches exhibit no abnormality.
3. TCP connections to a VIP (Virtual IP) may encounter intermittent timeouts, and traceroute probes to debug the issue may be blocked by the load balancers.
4. Load may not be balanced among a group of ECMP (Equal Cost Multi Path) links; the network administrators do not know if this issue is due to flow size differences or a deeper problem (e.g., a poor hash function).



ECMP imbalance

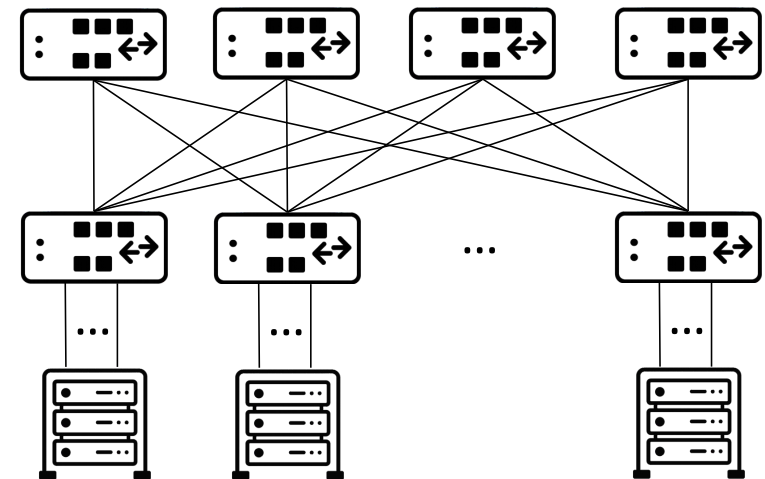
It has been demonstrated that with a 27K-host fat-tree and a randomized matrix, ECMP wastes an average of 61% of bisection bandwidth



ECMP imbalance

It has been demonstrated that with a 27K-host fat-tree and a randomized matrix, ECMP wastes an average of 61% of bisection bandwidth

- ECMP drawbacks:
 - Static and oblivious to link utilization
 - Can causes long-term flow collisions



Hedera: Dynamic Flow Scheduling for Data Center Networks

Hedera: Dynamic Flow Scheduling for Data Center Networks

- A solution proposed to improve ECMP
- Built to work on top of an SDN network and tested with the OpenFlow protocol
- Work presented at USENIX NSDI 2010 (San Josè)

Mohammad Al-Fares* Sivasankar Radhakrishnan*
Barath Raghavan† Nelson Huang* Amin Vahdat*
**{malfares, sivasankar, nhuang, vahdat}@cs.ucsd.edu †barath@cs.williams.edu*

**Department of Computer Science and Engineering †Department of Computer Science
University of California, San Diego Williams College*

Hedera: Dynamic Flow Scheduling for Data Center Networks

Hedera: Dynamic Flow Scheduling for Data Center Networks

Mohammad Al-Fares* Sivasankar Radhakrishnan*
Barath Raghavan† Nelson Huang* Amin Vahdat*
**{malfares, sivasankar, nhuang, vahdat}@cs.ucsd.edu †barath@cs.williams.edu*

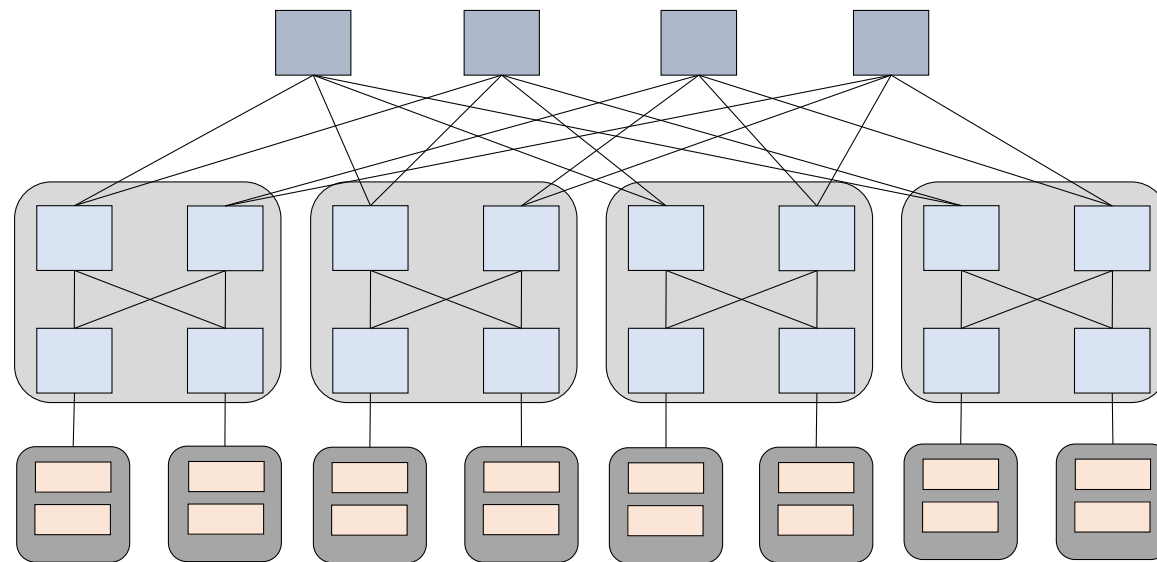
**Department of Computer Science and Engineering †Department of Computer Science
University of California, San Diego Williams College*

Problem Statement:

Given a dynamic traffic matrix of flow demands, how do you find paths that maximize network bisection bandwidth?

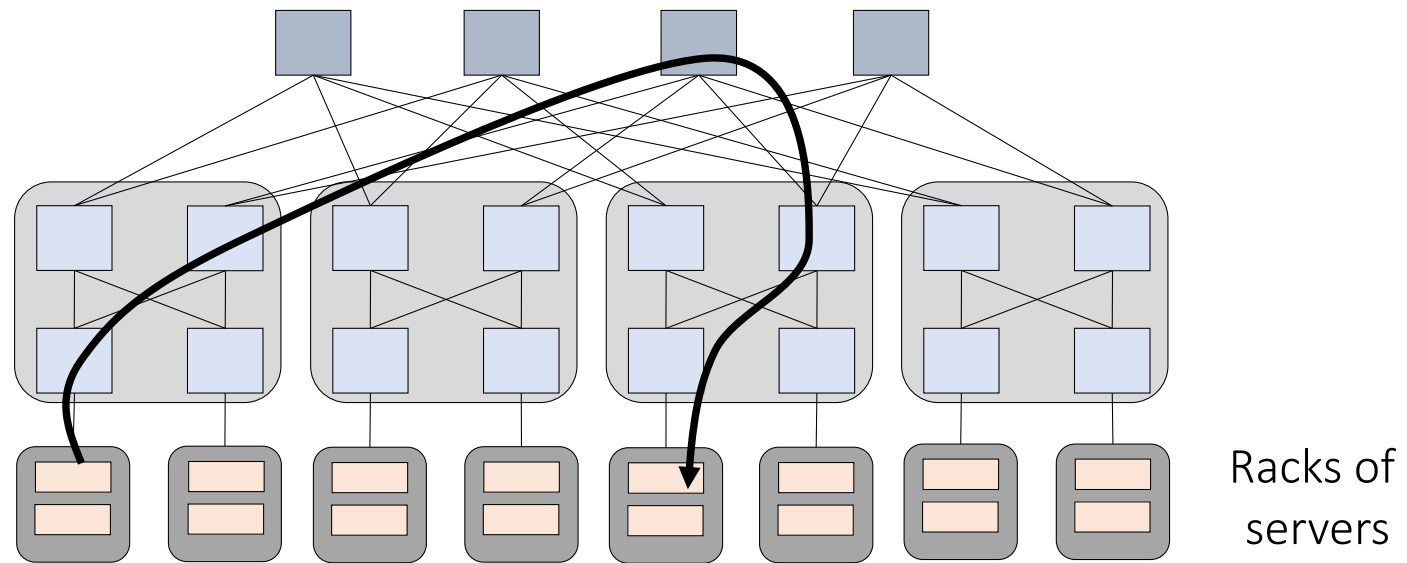
A traffic matrix is a two-dimensional matrix with its ij -th element (t_{ij}) determining the amount of traffic sourcing from a node i and exiting node j

Solving collisions with flow scheduling

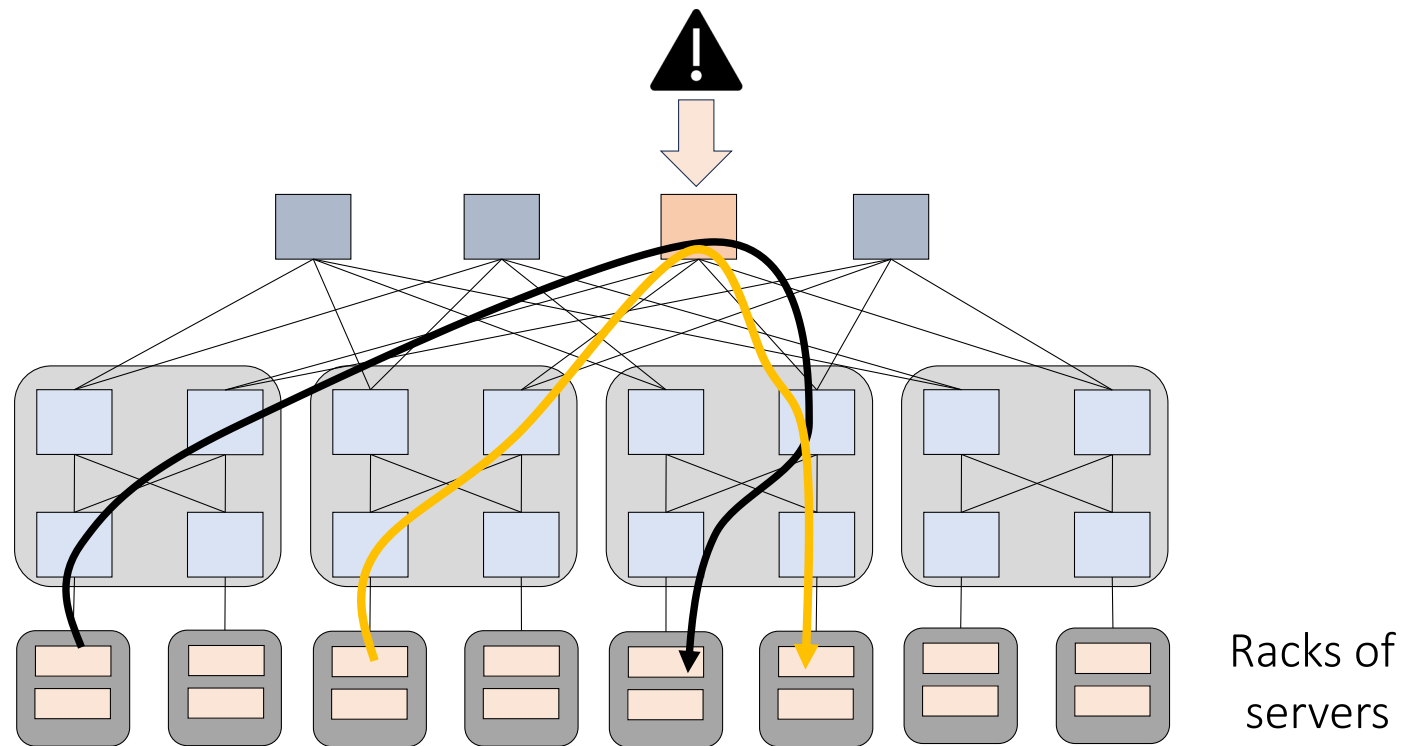


Racks of
servers

Solving collisions with flow scheduling

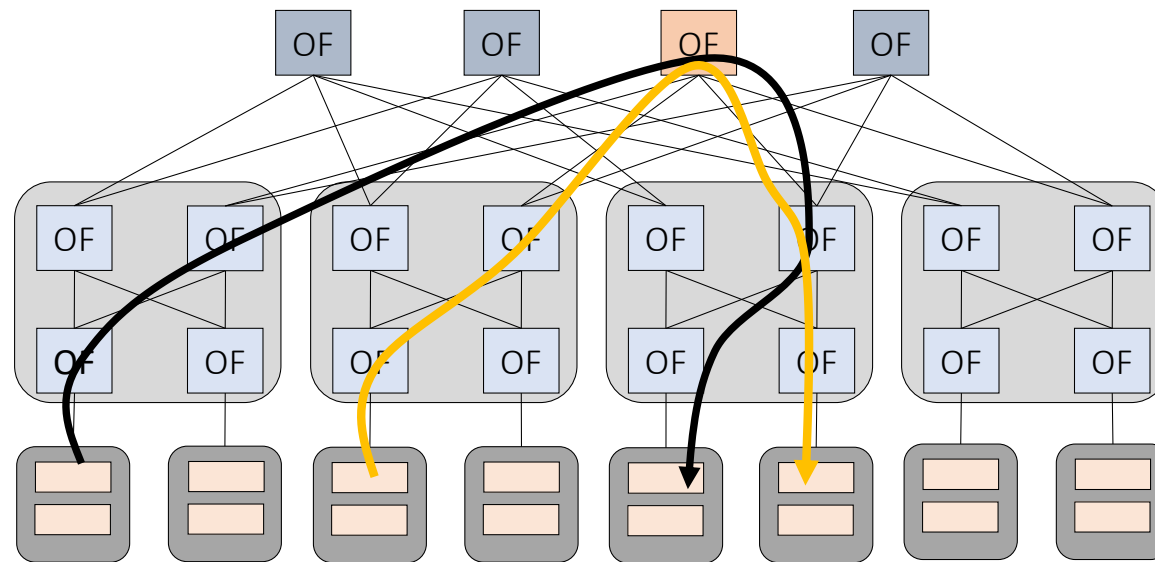


Solving collisions with flow scheduling



Solving collisions with flow scheduling

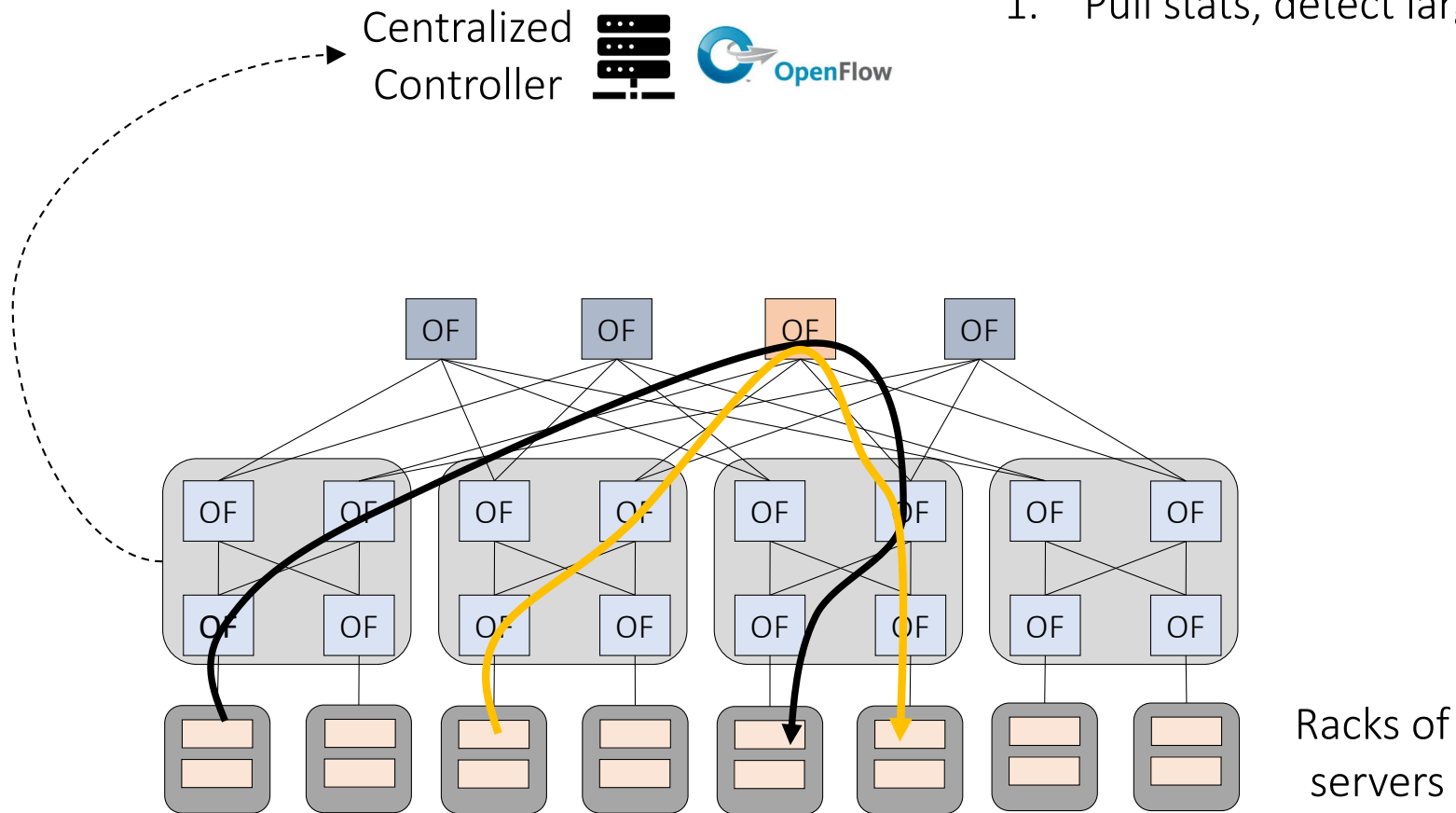
Centralized
Controller



Racks of
servers

Solving collisions with flow scheduling

1. Pull stats, detect large flows



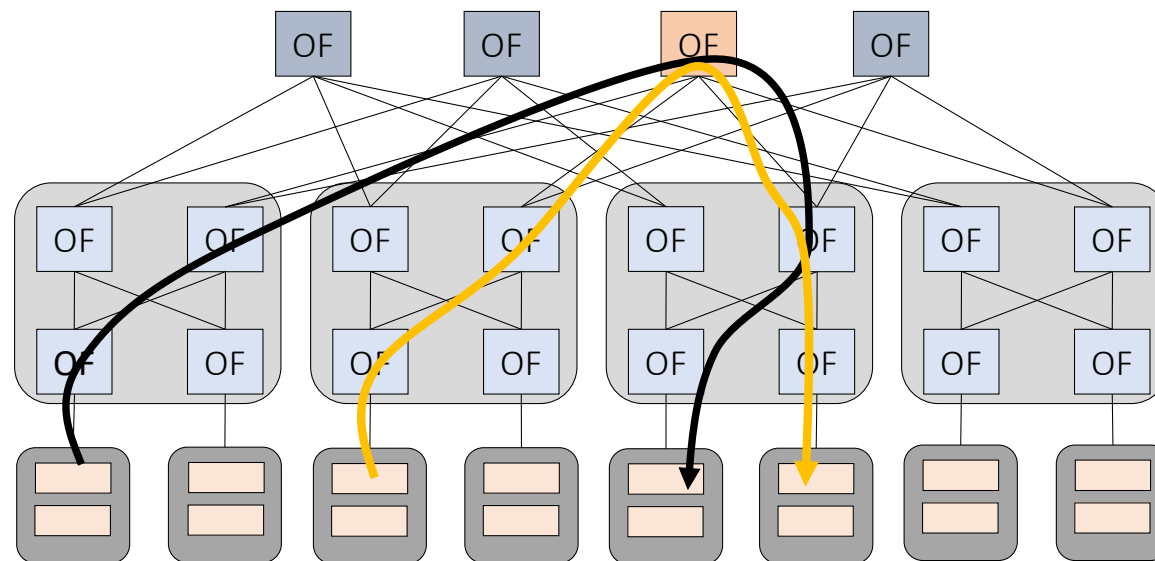
Solving collisions with flow scheduling



Centralized
Controller



1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement



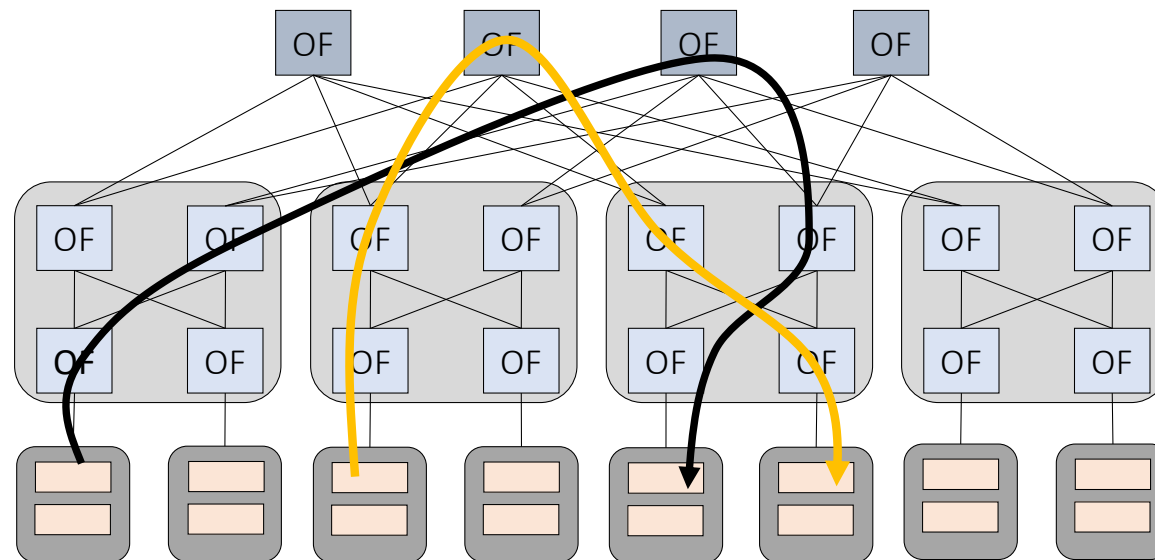
Racks of
servers

Solving collisions with flow scheduling

Centralized
Controller



1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows



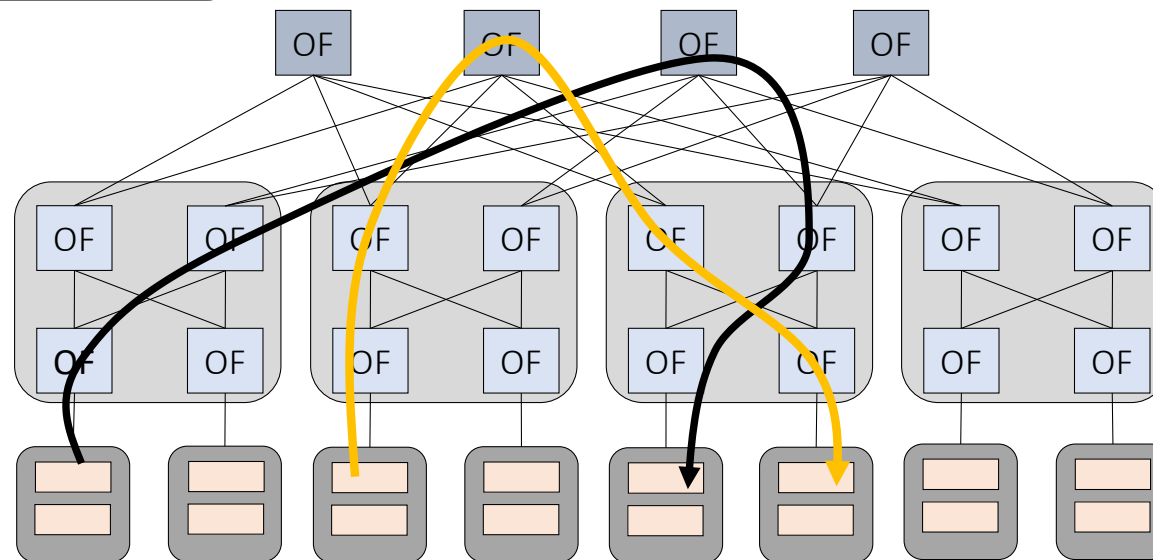
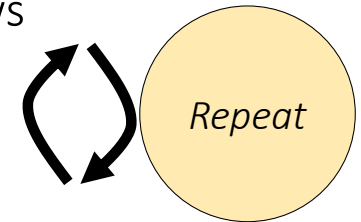
Racks of
servers

Solving collisions with flow scheduling

Centralized
Controller 

1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows

The main idea is to schedule only elephant flows and leave ECMP to deal with others



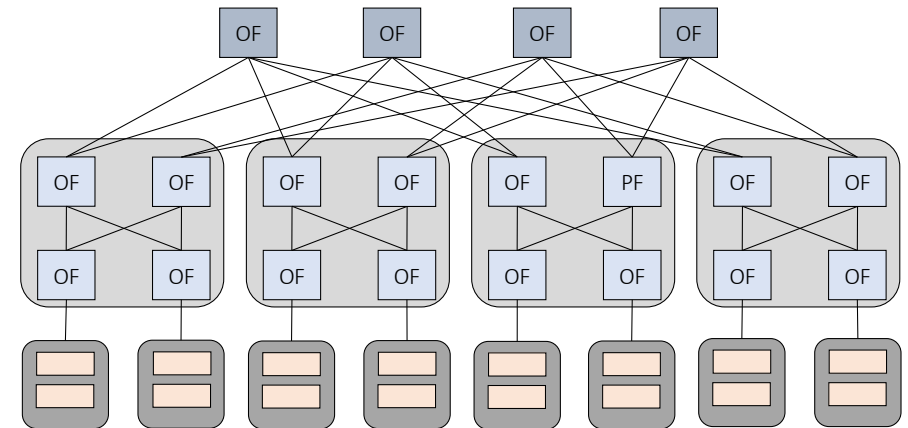
Racks of
servers

Solving collisions with flow scheduling

- Pull switch counters and use a pre-defined threshold to decide if a flow is large or not
- Why focussing only on heavy flows?

1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows

 Centralized
Controller



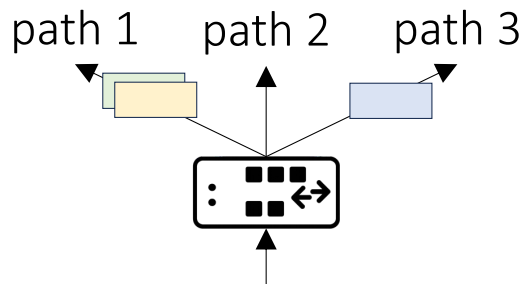
Solving collisions with flow scheduling

- Pull switch counters and use a pre-defined threshold to decide if a flow is large or not

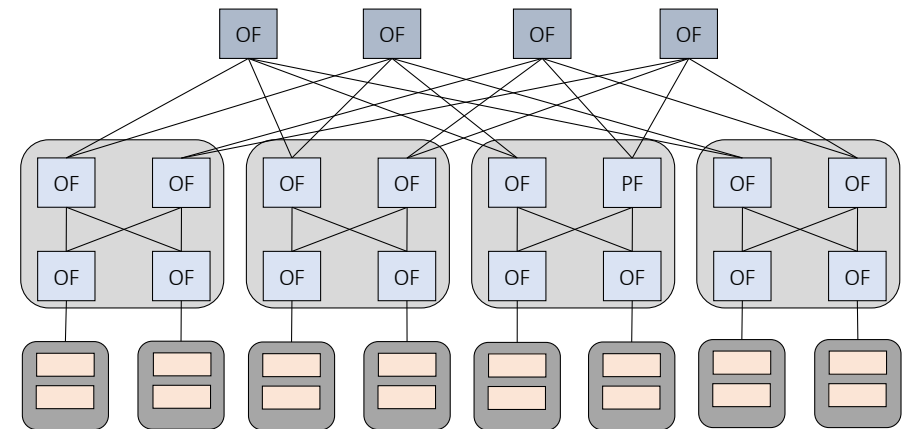
1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows

- Why focussing only on heavy flows?

ECMP is good enough for short flows as it spreads them over the multiple available paths!



 Centralized Controller

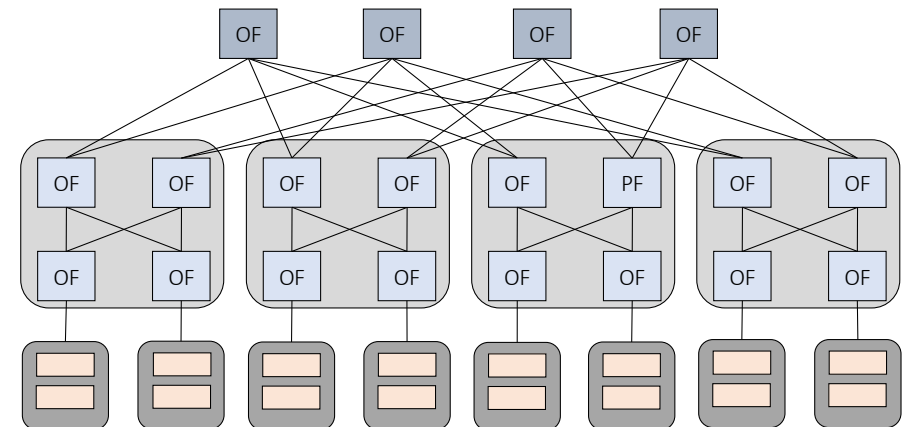


Solving collisions with flow scheduling

- Hedera *complements* ECMP!
 - Default forwarding uses ECMP
 - Hedera schedules large flows that cause bisection bandwidth problems

1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows

 Centralized
Controller

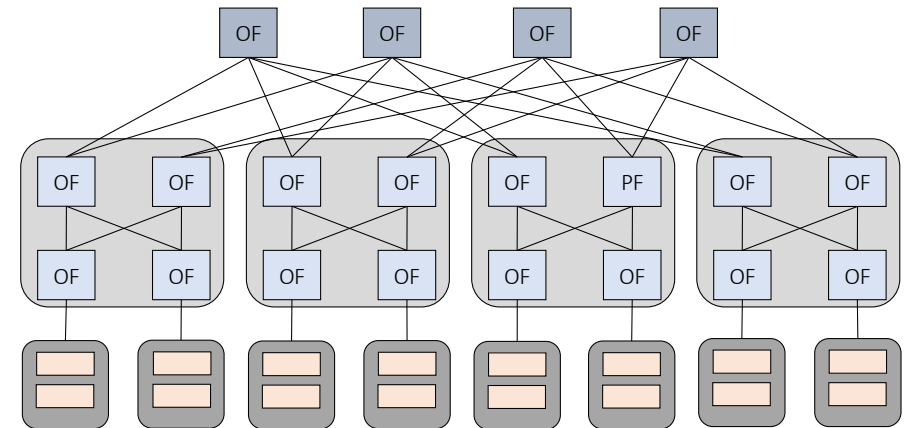


Solving collisions with flow scheduling

- If the controller gets counters continuously then it has the flow rates
- Can we consider that we have flow demands *for free*?

1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows

 Centralized
Controller



Solving collisions with flow scheduling

- If the controller gets counters continuously then it has the flow rates

- Can we consider that we have flow demands *for free*?

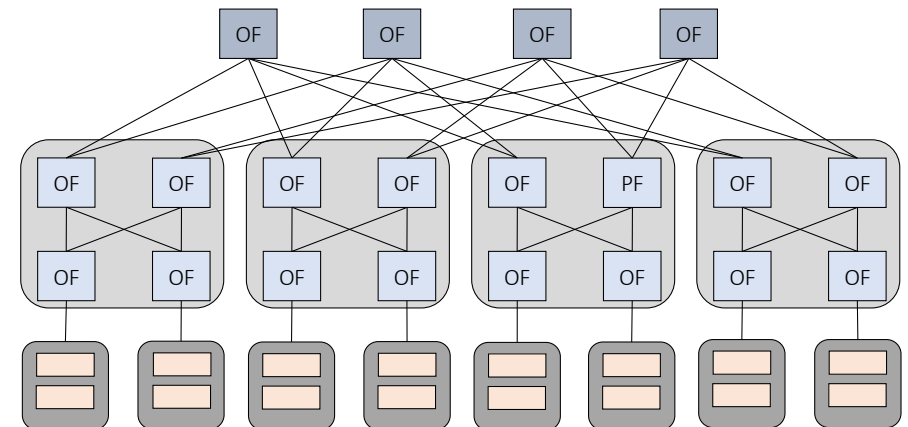
Nope ☹️

- Current flow rates are a poor indicator of flow demand
 - Network could be the bottleneck

1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows

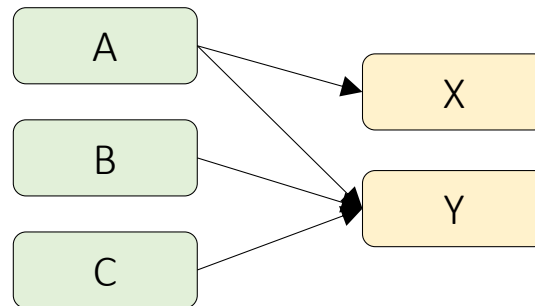


Centralized
Controller



Solving collisions with flow scheduling

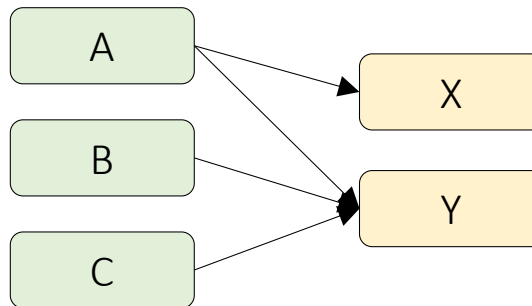
- Hedera's approach to compute flow demands: what would this flow get if the network was not a bottleneck?



Solving collisions with flow scheduling

- Hedera's approach to compute flow demands: what would this flow get if the network was not a bottleneck?

Sender	Flows	Share
A	2	1/2
B	1	1
C	1	1

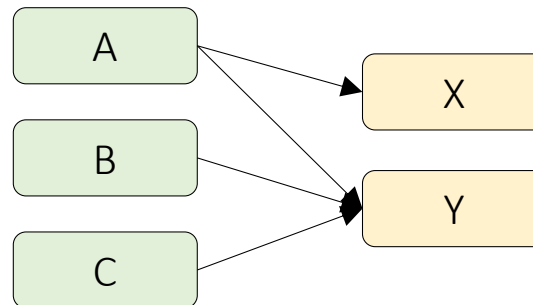


Solving collisions with flow scheduling

- Hedera's approach to compute flow demands: what would this flow get if the network was not a bottleneck?

Flow	Estimate
A to X	1/2
A to Y	1/2
B to Y	1
C to Y	1

Sender	Flows	Share
A	2	1/2
B	1	1
C	1	1

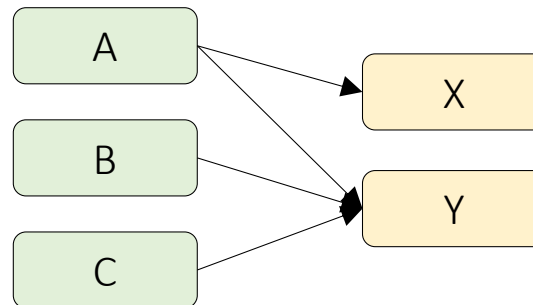


Solving collisions with flow scheduling

- Hedera's approach to compute flow demands: what would this flow get if the network was not a bottleneck?

Flow	Estimate
A to X	1/2
A to Y	1/2
B to Y	1
C to Y	1

Sender	Flows	Share
A	2	1/2
B	1	1
C	1	1

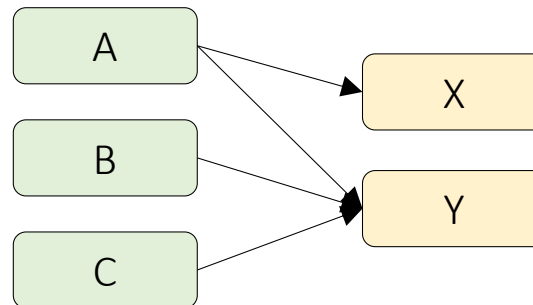


Receiver	Flows	Share
X	1	1
Y	3	1/3

Solving collisions with flow scheduling

- Hedera's approach to compute flow demands: what would this flow get if the network was not a bottleneck?

Sender	Flows	Share
A	2	1/2
B	1	1
C	1	1



Flow	Estimate
A to X	1/2
A to Y	1/2
B to Y	1
C to Y	1

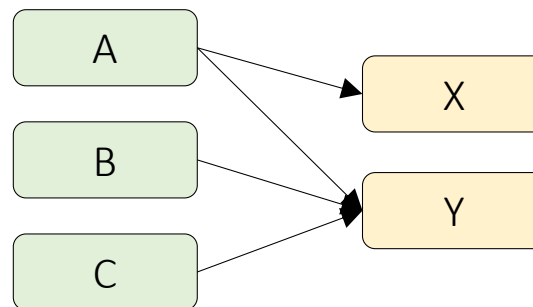
Receiver	Flows	Share
X	1	1
Y	3	1/3

Solving collisions with flow scheduling

- Hedera's approach to compute flow demands: what would this flow get if the network was not a bottleneck?

Flow	Estimate
A to X	1/2
A to Y	1/2 1/3
B to Y	1 1/3
C to Y	1 1/3

Sender	Flows	Share
A	2	1/2
B	1	1
C	1	1

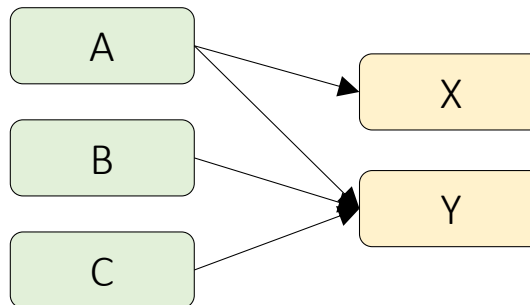


Receiver	Flows	Share
X	1	1
Y	3	1/3

Solving collisions with flow scheduling

- Hedera's approach to compute flow demands: what would this flow get if the network was not a bottleneck?

Sender	Flows	Share
A	2	1/2
B	1	1
C	1	1



Flow	Estimate
A to X	1/2
A to Y	1/2 1/3
B to Y	1 1/3
C to Y	1 1/3

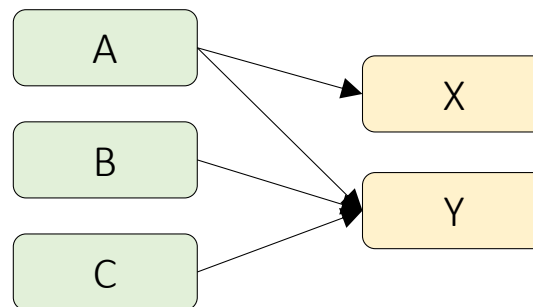
Receiver	Flows	Share
X	1	1
Y	3	1/3

Solving collisions with flow scheduling

- Hedera's approach to compute flow demands: what would this flow get if the network was not a bottleneck?

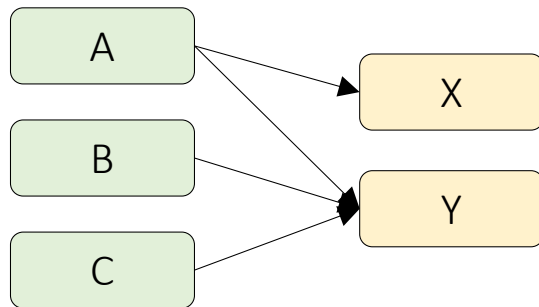
Flow	Estimate
A to X	1/2 2/3
A to Y	1/2 1/3
B to Y	1 1/3
C to Y	1 1/3

Sender	Flows	Share
A	2	1/2
B	1	1
C	1	1



Receiver	Flows	Share
X	1	1
Y	3	1/3

Solving collisions with flow scheduling

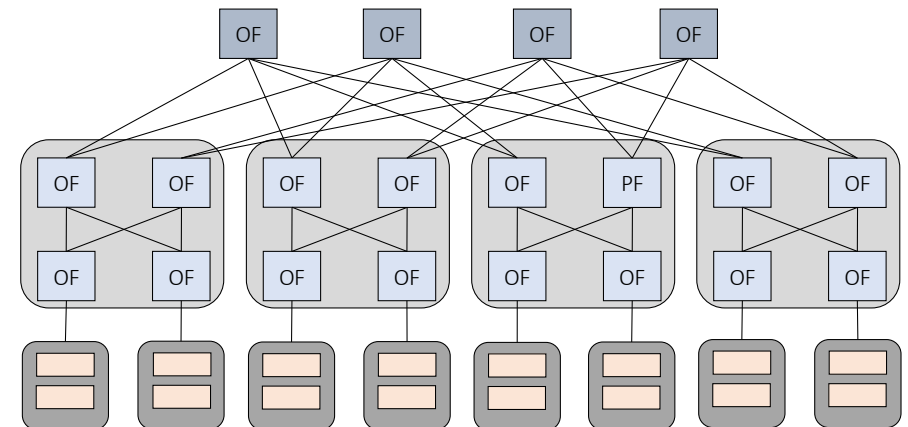


Flow	Estimate
A to X	2/3
A to Y	1/3
B to Y	1/3
C to Y	1/3

1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows



Centralized
Controller



Solving collisions with flow scheduling

- Two different policies proposed

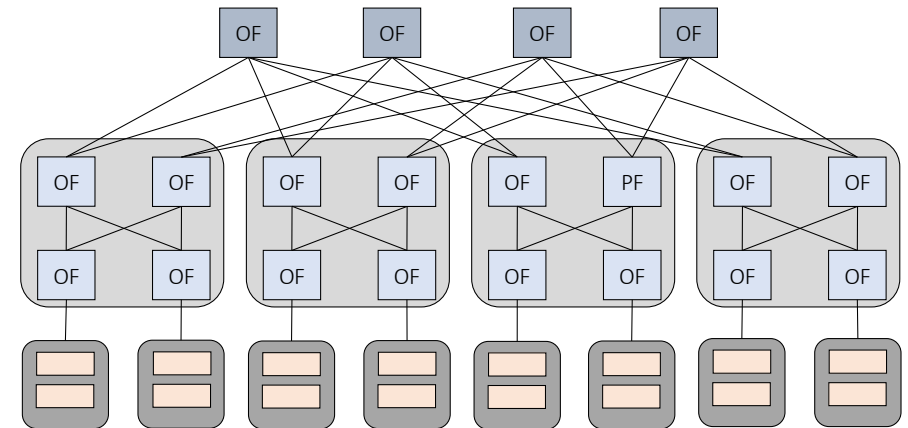
1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows

- **Global First-Fit**

- New flow detected, linearly search all possible paths from source to destination
- Place flow on first path whose component links can fit that flow
- Flows placed upon detection, are not moved
- Once flow ends, entries time out



Centralized
Controller



Solving collisions with flow scheduling

- Two different policies proposed

- **Global First-Fit**

- New flow detected, linearly search all possible paths from source to destination
- Place flow on first path whose component links can fit that flow
- Flows placed upon detection, are not moved
- Once flow ends, entries time out

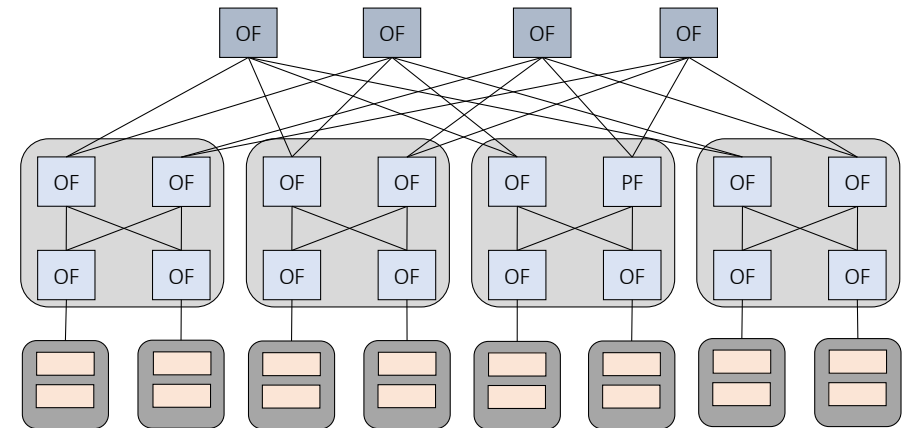
- **Simulated Annealing**

- Probabilistic search for good solutions that maximize bisection bandwidth

1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows



Centralized
Controller

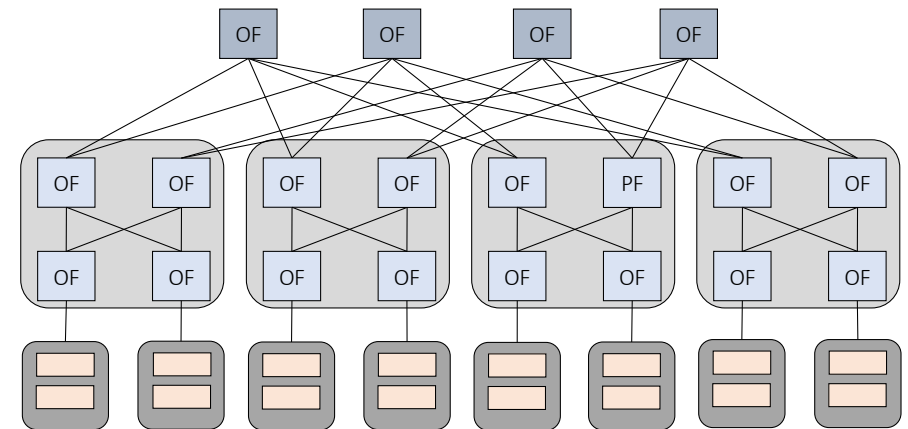


Solving collisions with flow scheduling

- In **Global First Fit**, a large flow can be re-routed immediately upon detection and is essentially pinned to its reserved links.
- **Simulated Annealing** waits for the next scheduling tick, uses previously computed flow placements to optimize the current placement.

1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows

 Centralized
Controller

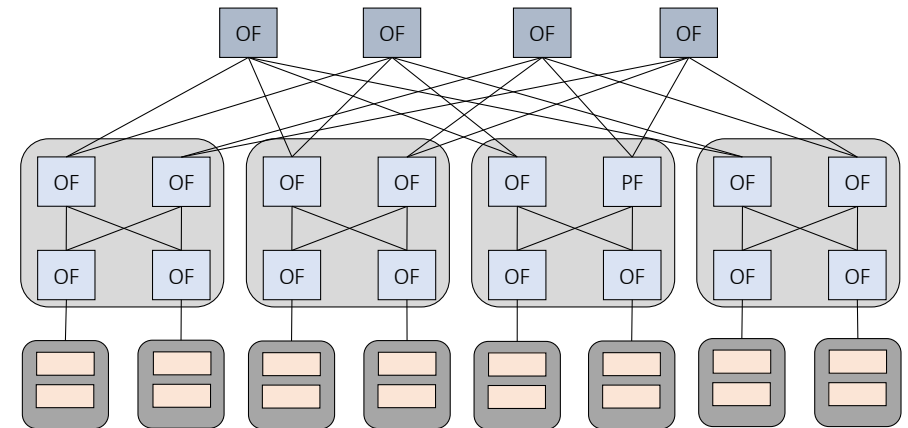


Solving collisions with flow scheduling

- Use the centralized controller to install appropriate routing rules into switches (implementation based on the OpenFlow protocol)

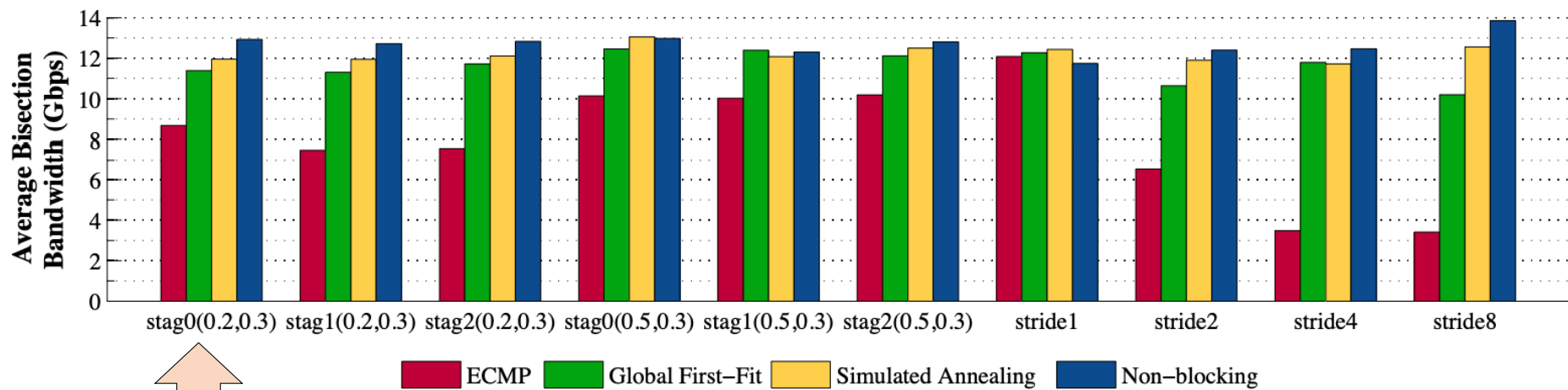
1. Pull stats, detect large flows
2. Compute flow demands
3. Compute placement
4. Place flows

 Centralized Controller



Does it work?

16 hosts testbed using a fat tree with 20 4-ports
OpenFlow switches



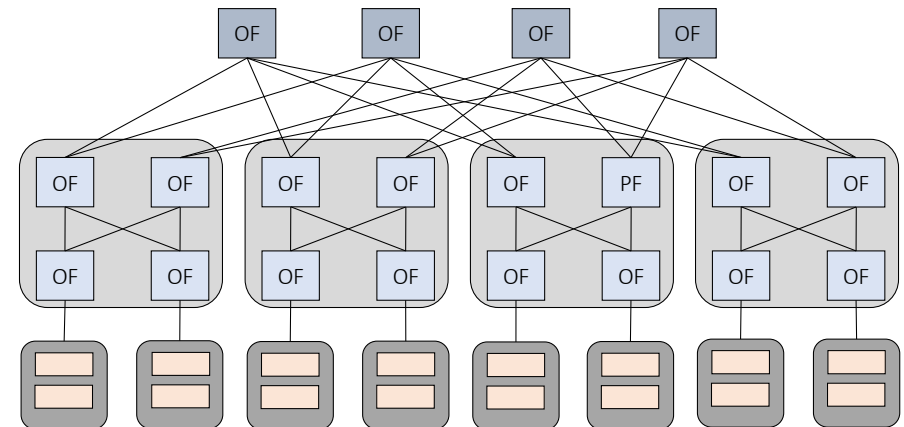
Different communication patterns

Ideal case

Solving collisions with flow scheduling

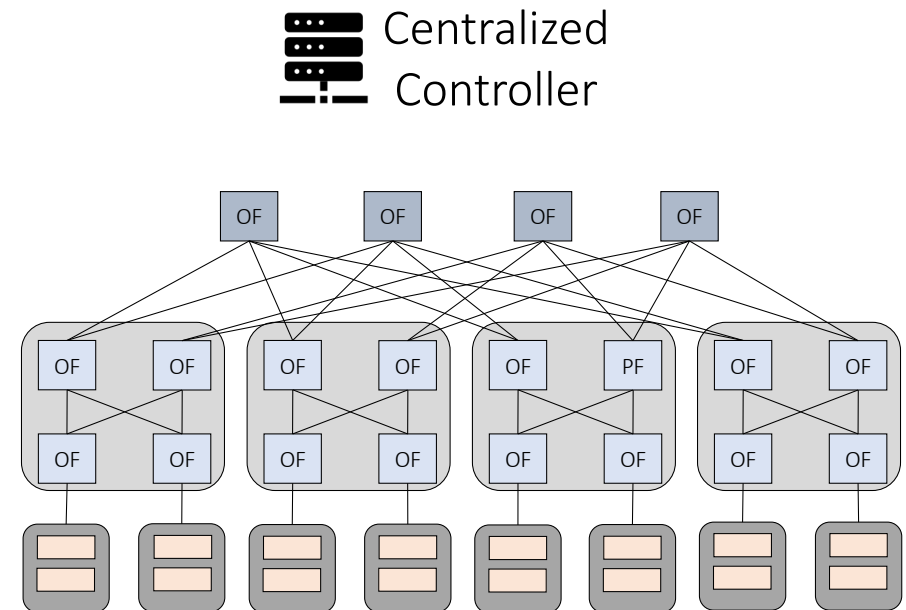
- Problems with Hedera?

 Centralized
Controller



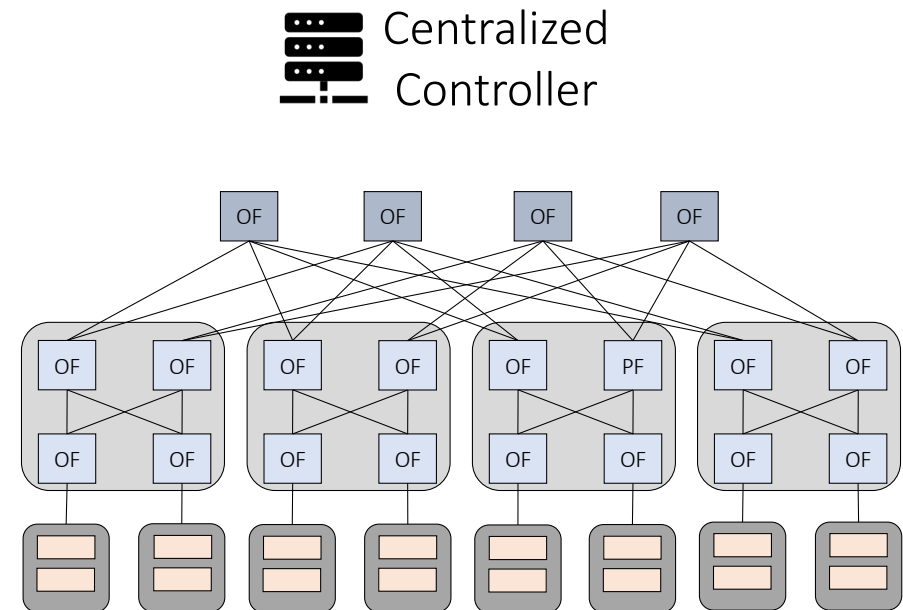
Solving collisions with flow scheduling

- Problems with Hedera?
 - Dynamic workloads might generate large flow turnover faster than control loop: scheduler will be continually chasing the traffic matrix
 - Can it scale to really large datacenters?



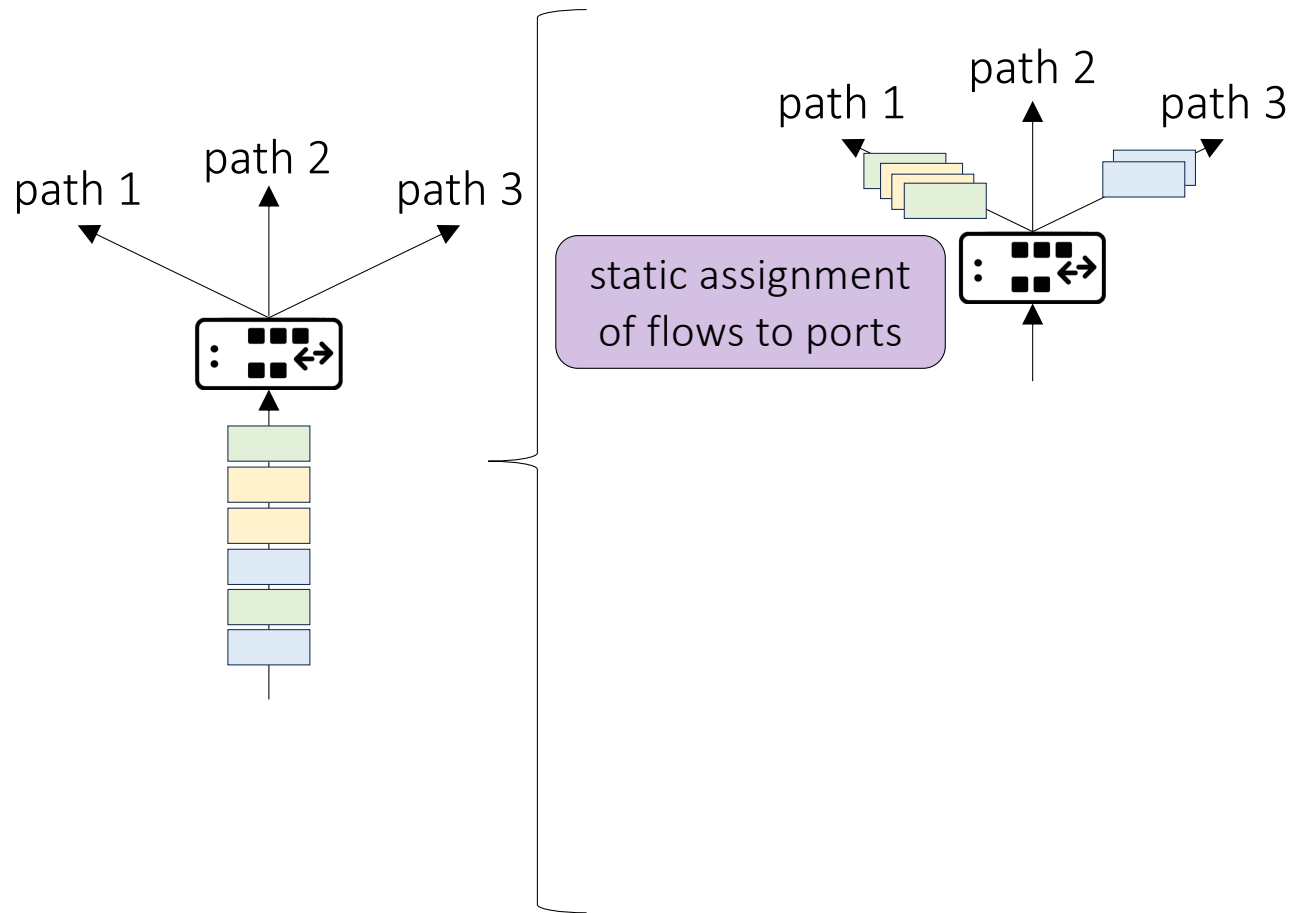
Solving collisions with flow scheduling

- If the problem is the centralized controller, can we get rid of it and implement load balancing back in the switches (as ECMP was doing)?
- This makes sense because datacenter traffic is very bursty and unpredictable
- Not much time available for the control loop



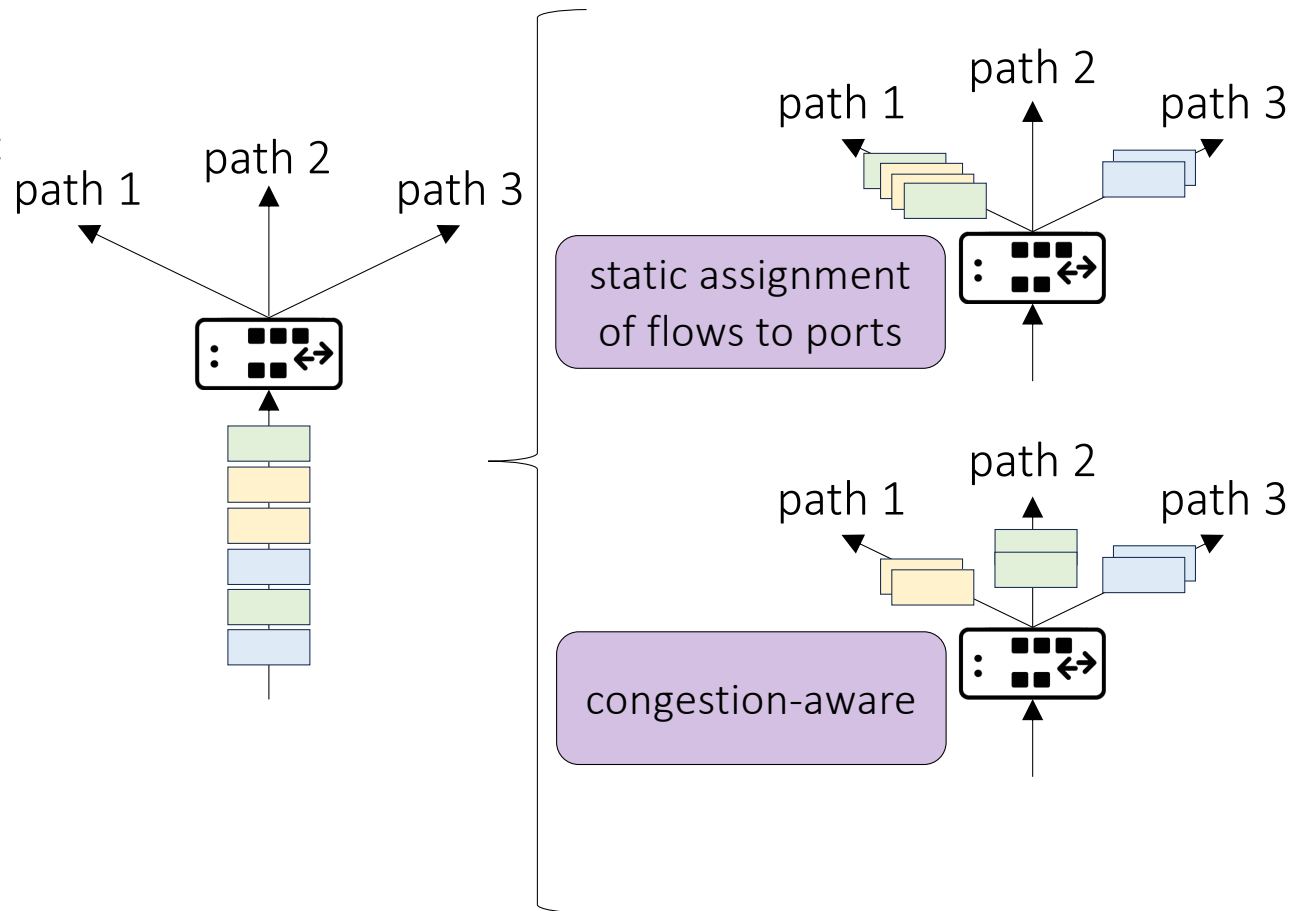
Are we back to ECMP?

- The problem with ECMP is that it is static



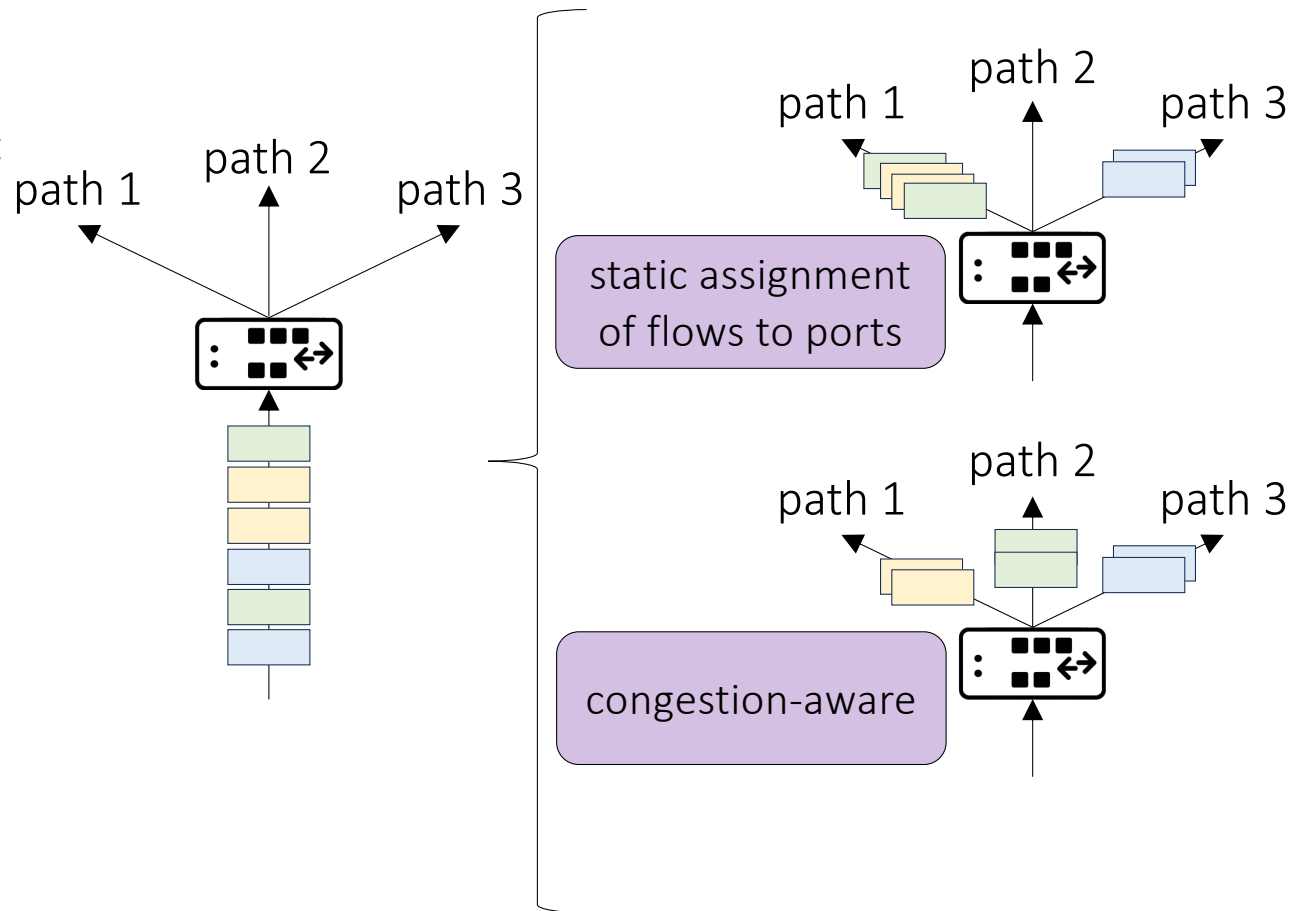
Congestion-aware load balancing

- The problem with ECMP is that it is static
- What we instead pick the least congested port?



Congestion-aware load balancing

- The problem with ECMP is that it is static
- What we instead pick the least congested port?
- What's the right granularity, though?



Looking for the *right* granularity

- Per-packet load balancing would be ideal but introduce packet-reordering which is bad for TCP
- Per-flow load balancing is fine but too coarse-grained to fully utilize the network
- What can we do?

Looking for the *right* granularity

Dynamic Load Balancing Without Packet Reordering

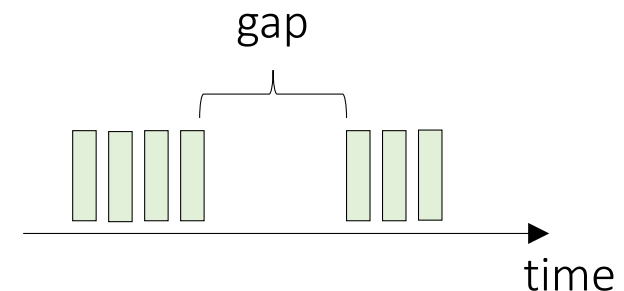
- Flowlet load balancing!
- Flowlets are bursts of packets from a flow that are separated by large enough gaps
- If the gap is *large enough* then it is possible to route the two flowlets to different paths without risking reordering

Srikanth Kandula
MIT
kandula@mit.edu

Dina Katabi
MIT
dk@mit.edu

Shantanu Sinha
Microsoft
shans@microsoft.com

Arthur Berger
MIT/Akamai
awberger@mit.edu



Let's put this altogether

HULA: Scalable Load Balancing Using Programmable Data Planes

Naga Katta*, Mukesh Hira†, Changhoon Kim‡, Anirudh Sivaraman*, Jennifer Rexford*

*Princeton University, †VMware, ‡Barefoot Networks, *MIT CSAIL

{nkatta, jrex}@cs.princeton.edu, mhira@vmware.com, chang@barefootnetworks.com, anirudh@csail.mit.edu

- A congestion-aware load balancing solution with flowlet switching
- Built to work on P4 programmable switches
- Work presented at ACM SOSR 2016 (Santa Clara)

Let's put this altogether

HULA: Scalable Load Balancing Using Programmable Data Planes

Naga Katta*, Mukesh Hira†, Changhoon Kim‡, Anirudh Sivaraman*, Jennifer Rexford*

*Princeton University, †VMware, ‡Barefoot Networks, *MIT CSAIL

{nkatta, jrex}@cs.princeton.edu, mhira@vmware.com, chang@barefootnetworks.com, anirudh@csail.mit.edu

- Challenges:
 - Datacenter networks are big. If we want to create a congestion-aware algorithm, how to keep track scalably of all the possible paths?
 - How to do this continuously (in a proactive manner)?
 - How to implement such an algorithm given the constraints of programmable switches?

Let's put this altogether

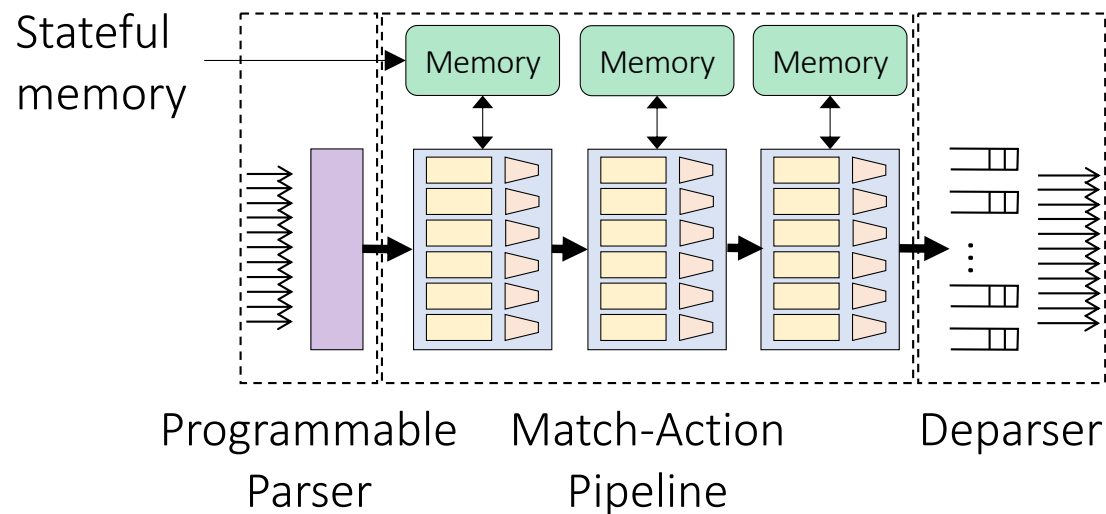
HULA: Scalable Load Balancing Using Programmable Data Planes

Naga Katta*, Mukesh Hira†, Changhoon Kim‡, Anirudh Sivaraman*, Jennifer Rexford*

*Princeton University, †VMware, ‡Barefoot Networks, *MIT CSAIL

{nkatta, jrex}@cs.princeton.edu, mhira@vmware.com, chang@barefootnetworks.com, anirudh@csail.mit.edu

- Challenges:
 - Datacenter networks are big. If we want to create a congestion-aware algorithm, how to keep track scalably of all the possible paths?
 - How to do this continuously (in a proactive manner)?
 - How to implement such an algorithm given the constraints of programmable switches?

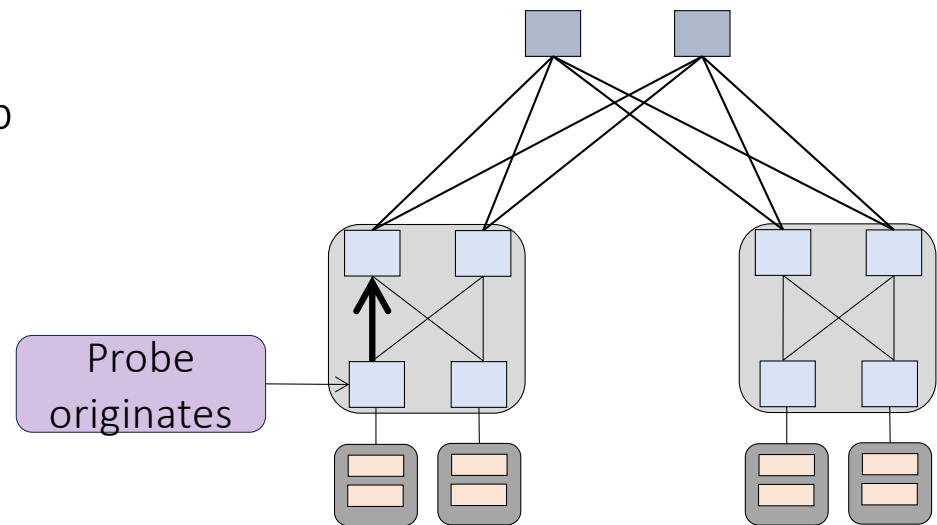


Hop-by-hop Utilization-aware Load-balancing Architecture (HULA)

- HULA switches exchange probes to propagate path utilization
 - Allows continuous monitoring of network status
- Each switch remembers *only* the best next hop
 - Scalable and topology-oblivious
- Split elephants to mice flows (flowlets)
 - Fine-grained load balancing

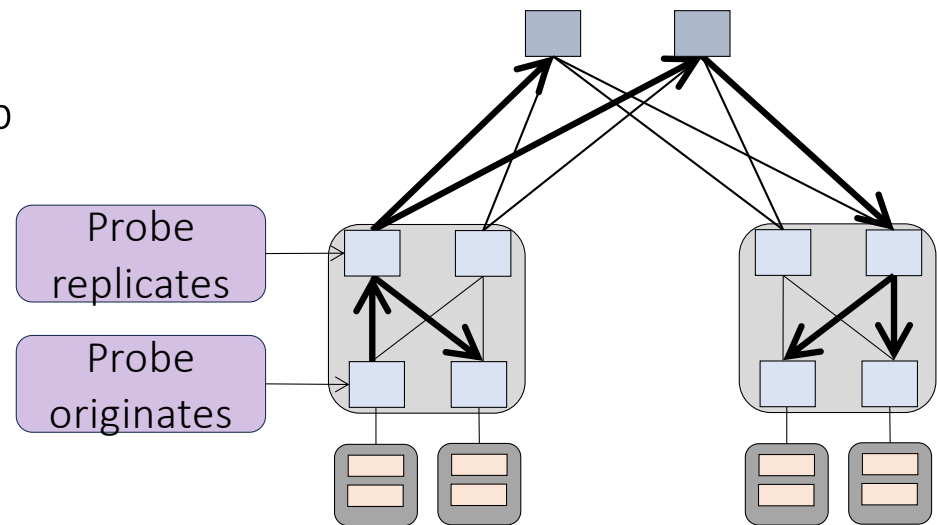
Hop-by-hop Utilization-aware Load-balancing Architecture (HULA)

- HULA switches exchange probes to propagate path utilization
 - Allows continuous monitoring of network status
- Each switch remembers *only* the best next hop
 - Scalable and topology-oblivious
- Split elephants to mice flows (flowlets)
 - Fine-grained load balancing



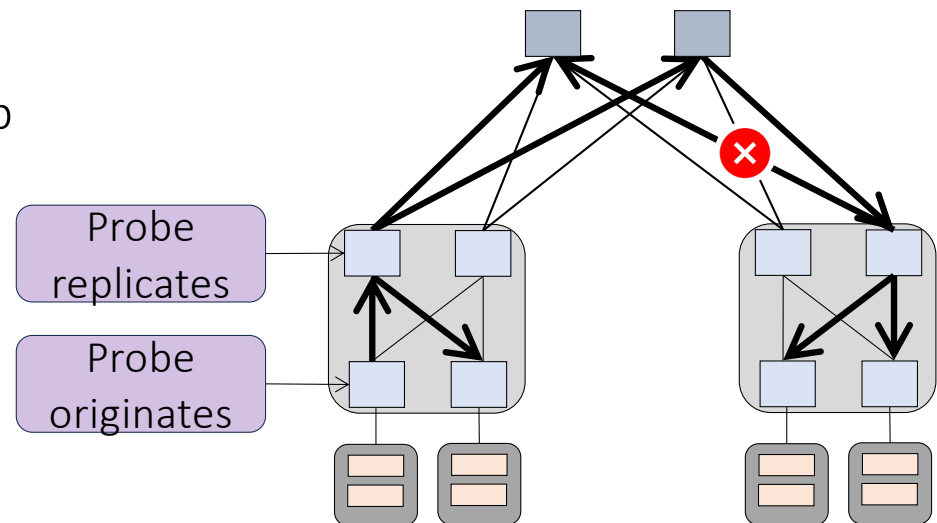
Hop-by-hop Utilization-aware Load-balancing Architecture (HULA)

- HULA switches exchange probes to propagate path utilization
 - Allows continuous monitoring of network status
- Each switch remembers *only* the best next hop
 - Scalable and topology-oblivious
- Split elephants to mice flows (flowlets)
 - Fine-grained load balancing



Hop-by-hop Utilization-aware Load-balancing Architecture (HULA)

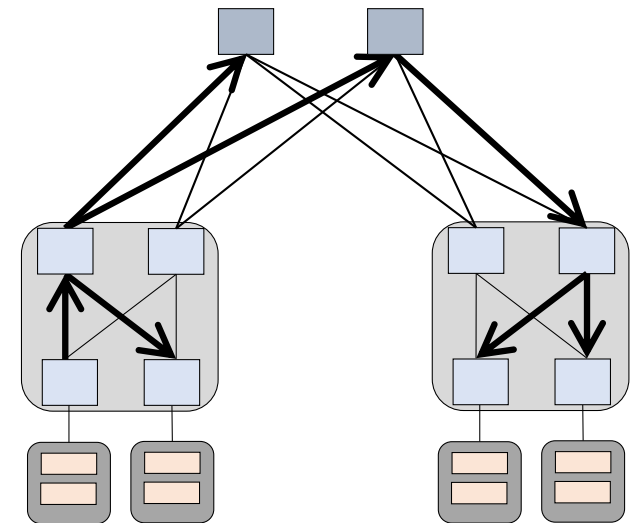
- HULA switches exchange probes to propagate path utilization
 - Allows continuous monitoring of network status
- Each switch remembers *only* the best next hop
 - Scalable and topology-oblivious
- Split elephants to mice flows (flowlets)
 - Fine-grained load balancing



Hop-by-hop Utilization-aware Load-balancing Architecture (HULA)

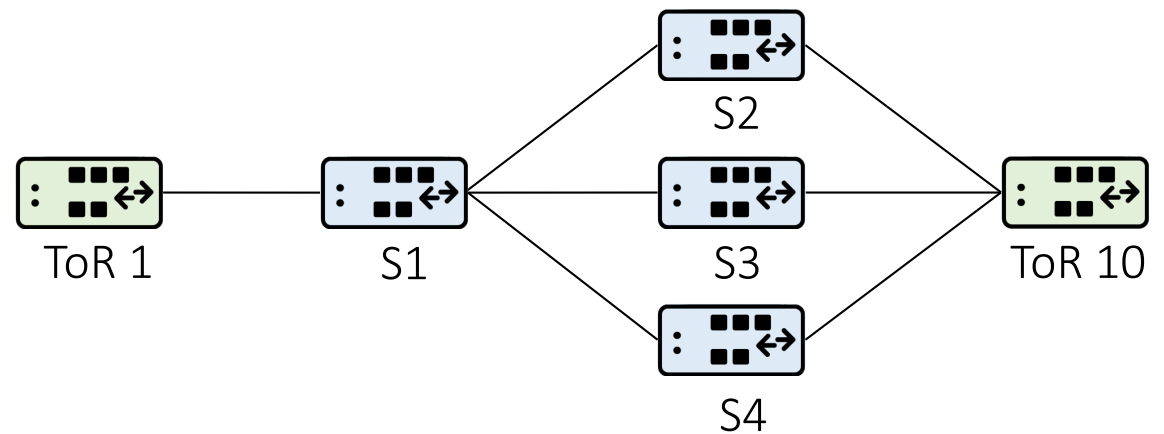
- A HULA probe is a minimum-sized IP packet, including the HULA header

Ver.	IHL	DSCP	ECN	Total Length	
IPID				Flags	Fragment Offest
TTL		Protocol		Header Checksum	
Source IP					
Destination IP					
HULA torID					HULA pathUtil



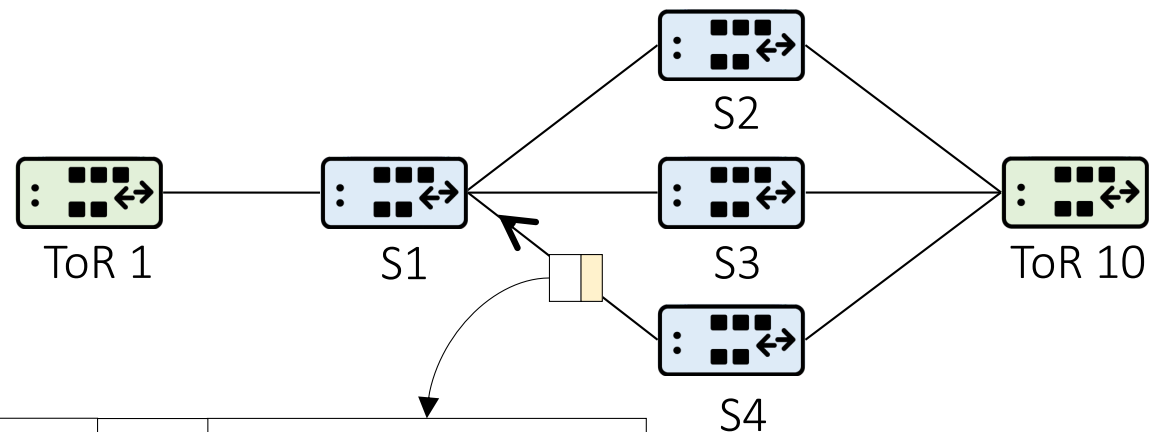
Congestion-aware routing

Idea: every ToR maintains a link utilization per port



Congestion-aware routing

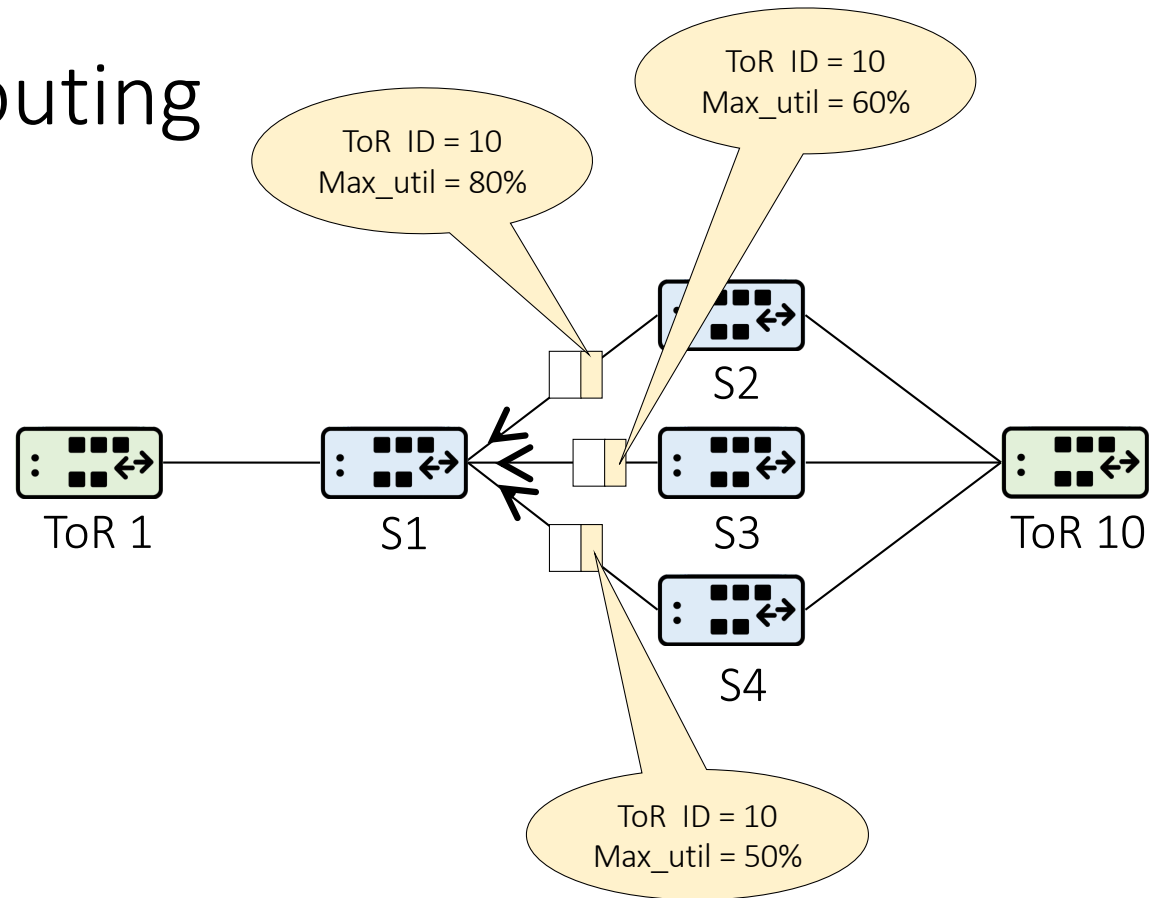
Idea: every ToR maintains a link utilization per port



Ver.	IHL	DSCP	ECN	Total Length	
IPID				Flags	Fragment Offest
TTL		Protocol		Header Checksum	
Source IP					
Destination IP					
HULA torID					HULA pathUtil

Congestion-aware routing

Idea: every ToR maintains a link utilization per port



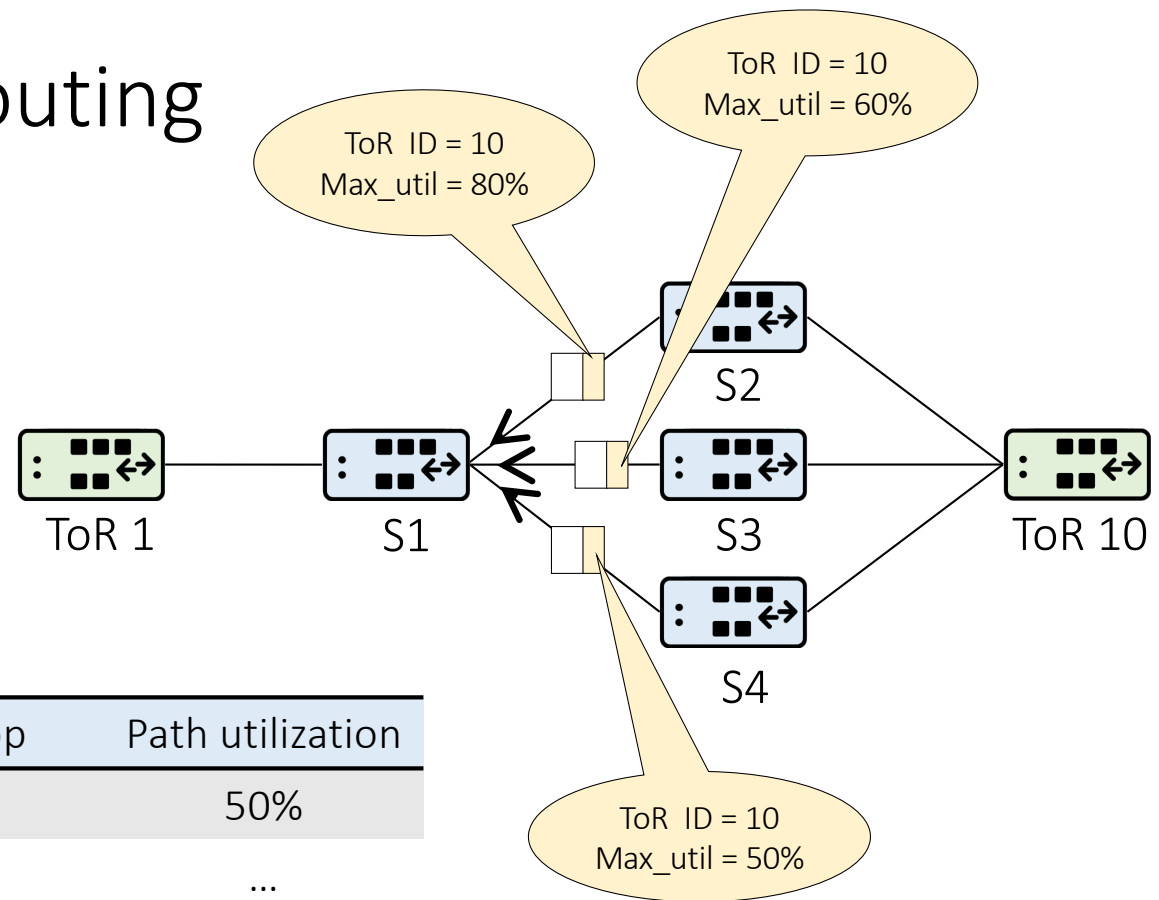
Congestion-aware routing

Idea: every ToR maintains a link utilization per port

S1 takes the minimum from among the probes given utilization and stores it in the local routing table

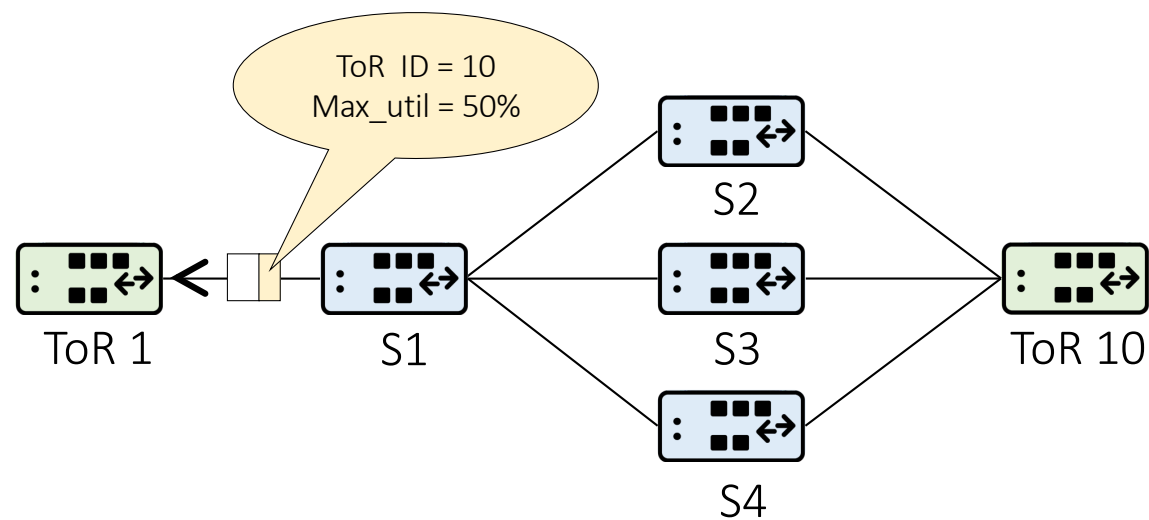
Destination	Best hop	Path utilization
ToR 10	S4	50%
...	...	...
...	...	...

S1 Best hop table



Congestion-aware routing

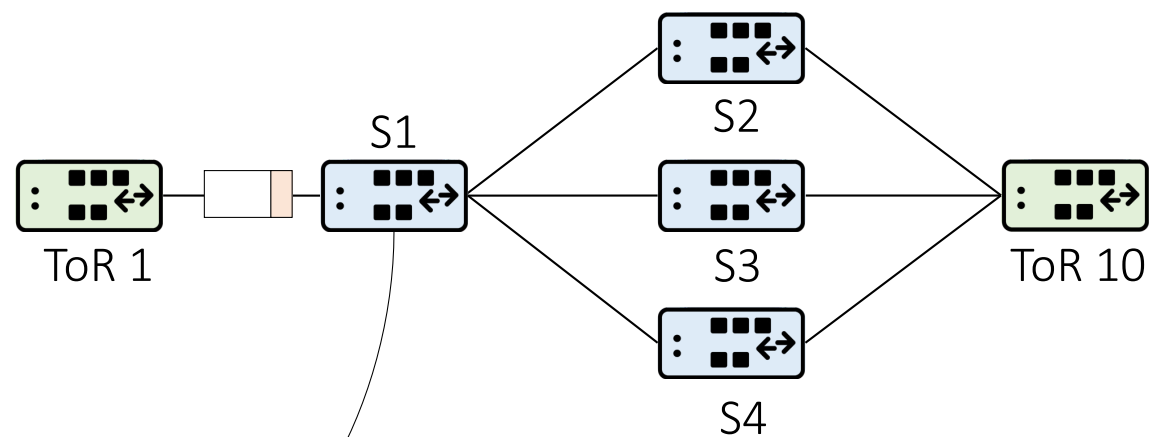
- S1 sends its view of the best path to upstream switches
- This is similar to distance vector information propagation
- Switches keep track of only the best next hop



Destination	Best hop	Path utilization
ToR 10	S4	50%
...	...	...
...	...	...

S1 Best hop table

HULA in action: from ToR 1 to ToR 10

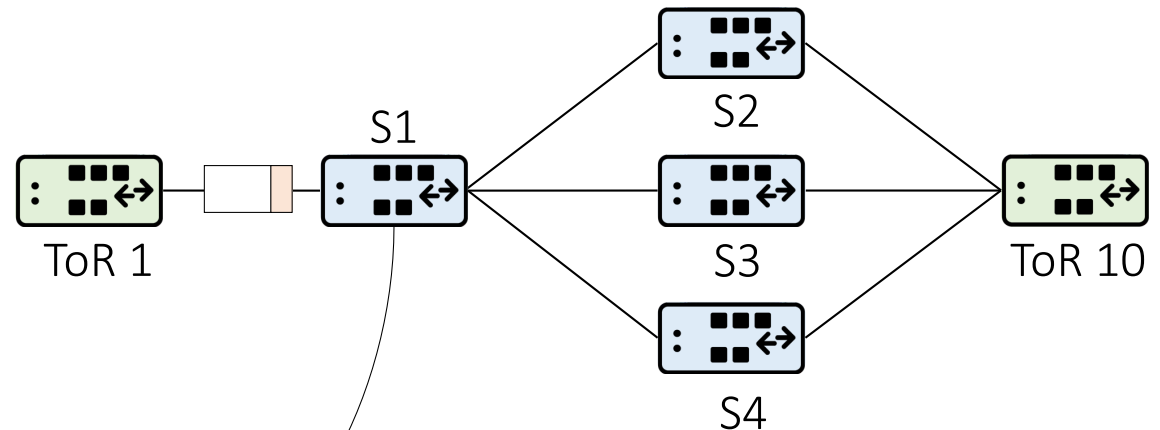


Destination	Best hop	Path utilization
ToR 10	S4	50%
...	...	...
...	...	...

S1 Best hop table

HULA in action: from ToR 1 to ToR 10

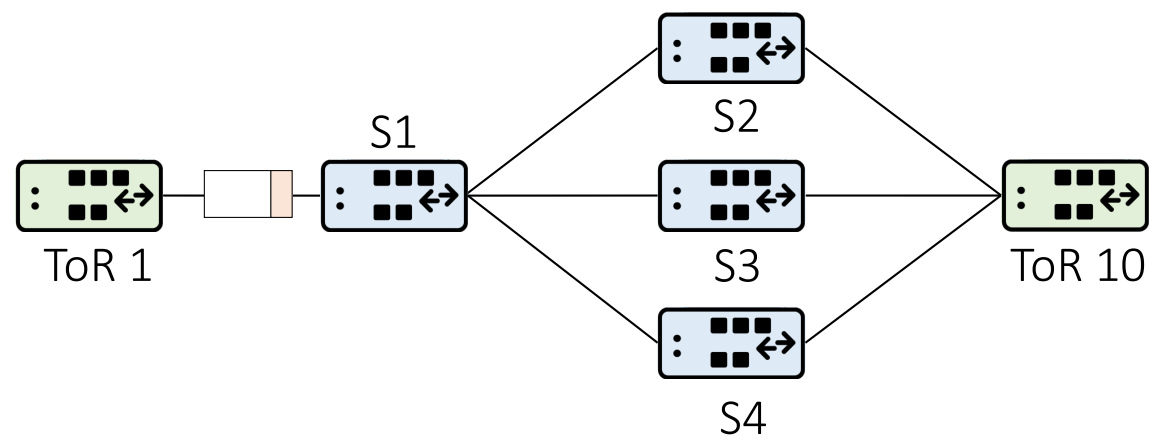
What would be the problem with this design?



Destination	Best hop	Path utilization
ToR 10	S4	50%
...	...	...
...	...	...

S1 Best hop table

HULA in action: from ToR 1 to ToR 10



Destination	Best hop	Path utilization
ToR 10	S4	50%
...	...	...
...	...	...

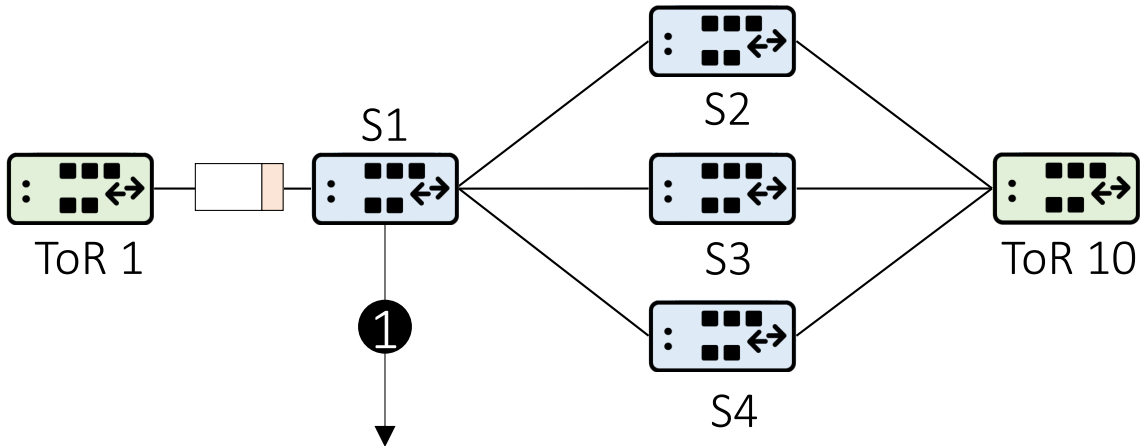
S1 Best hop table

Destination	Timestamp	Next hop

S1 flowlet cache

HULA in action: from ToR 1 to ToR 10

1. Check flowlet cache



Destination	Best hop	Path utilization
ToR 10	S4	50%
...	...	...
...	...	...

S1 Best hop table

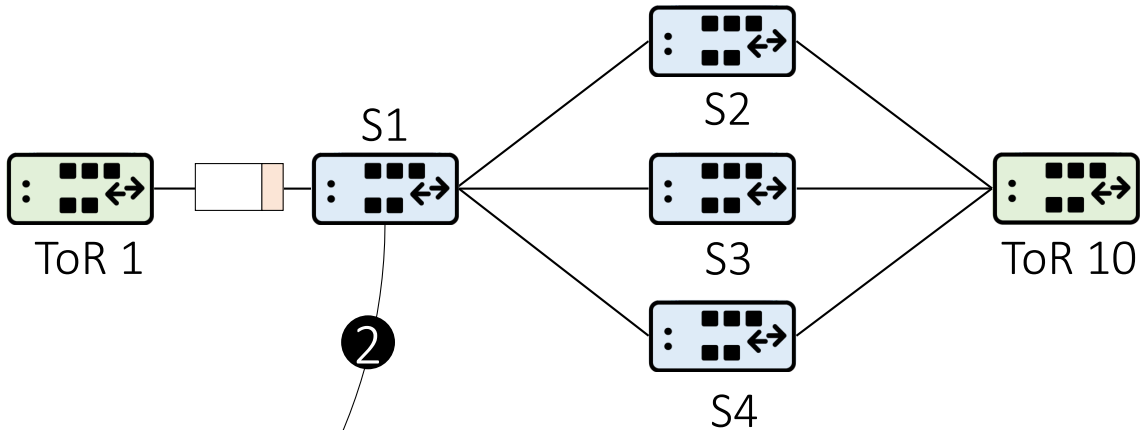
Destination	Timestamp	Next hop

S1 flowlet cache

HULA in action: from ToR 1 to ToR 10

1. Check flowlet cache

2. If miss, then check best hop table



Destination	Best hop	Path utilization
ToR 10	S4	50%
...	...	...
...	...	...

S1 Best hop table

Destination	Timestamp	Next hop

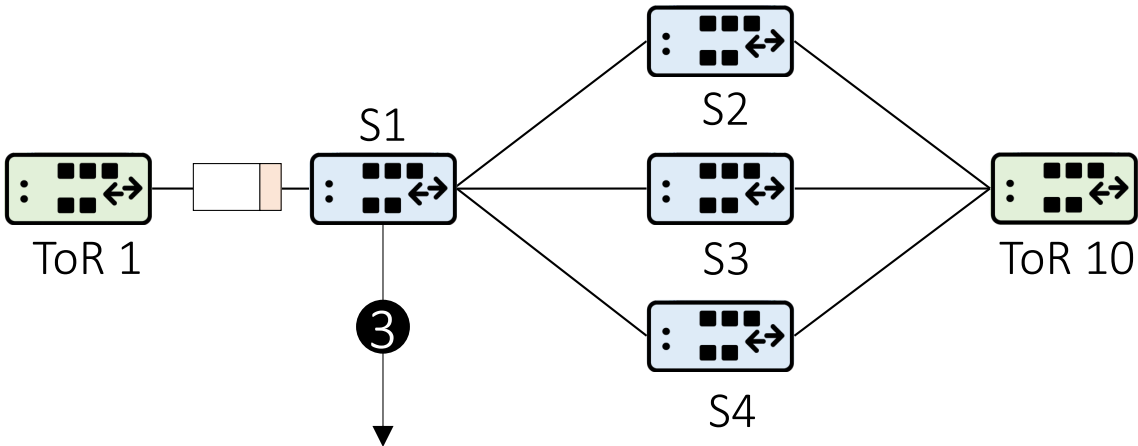
S1 flowlet cache

HULA in action: from ToR 1 to ToR 10

1. Check flowlet cache

2. If miss, then check best hop table

3. Record the entry in the cache



Destination	Best hop	Path utilization
ToR 10	S4	50%
...	...	...
...	...	...

S1 Best hop table

Destination	Timestamp	Next hop
ToR 10	1	S4

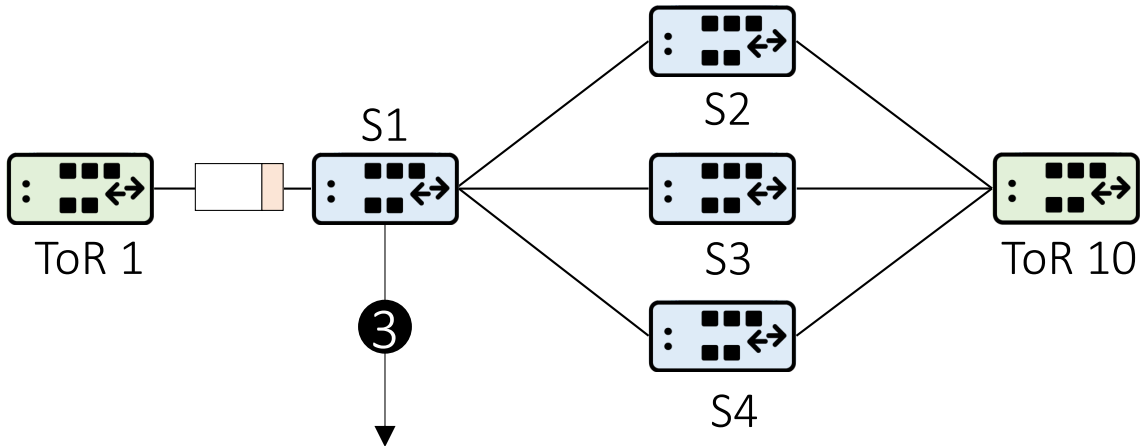
S1 flowlet cache

HULA in action: from ToR 1 to ToR 10

1. Check flowlet cache

2. If miss, then check best hop table

3. Record the entry in the cache



Destination	Best hop	Path utilization
ToR 10	S4	50%
...	...	
...	...	

S1 Best hop table

Destination	Timestamp	Next hop
ToR 10	1	S4
...	...	
...	...	

S1 flowlet cache

entry is cleared upon timeout expiration

P4 for HULA

HULA can be described in P4!

```
header_type hula_header {
    fields{
        dst_tor : 24;
        path_util : 8;
    }
}

header_type metadata{
    fields{
        nxt_hop : 8;
        self_id : 32;
        dst_tor : 32;
    }
}

control ingress {
    apply(get_dst_tor)
    apply(hula_logic)
    if(ipv4.protocol == PROTO_HULA){
        apply(hula_mcast);
    }
    else if(metadata.dst_tor
            == metadata.self_id) {
        apply(send_to_host);
    }
}

1 action hula_logic{
2     if(ipv4_header.protocol == IP_PROTOCOLS_HULA){
3         /*HULA Probe Processing
4         if(hula_hdr.path_util < tx_util)
5             hula_hdr.path_util = tx_util;
6         if(hula_hdr.path_util < min_path_util[hula_hdr.dst_tor] ||
7             curr_time - update_time[dst_tor] > KEEP_ALIVE_THRESH)
8             {
9                 min_path_util[dst_tor] = hula_hdr.path_util;
10                best_hop[dst_tor] = metadata.in_port;
11                update_time[dst_tor] = curr_time;
12            }
13        hula_header.path_util = min_path_util[hula_hdr.dst_tor];
14    }
15    else { /*Flowlet routing of data */
16        if(curr_time - flowlet_time[flow_hash]> FLOWLET_TOUT) {
17            flowlet_hop[flow_hash] = best_hop[metadata.dst_tor];
18        }
19        metadata.nxt_hop = flowlet_hop[flow_hash];
20        flowlet_time[flow_hash] = curr_time;
21    }
22 }
```


About HULA

- What happens if a link fail?
- Shall we run the probes more or less frequently?
- What's the right flowlet timeout?

About HULA

- What happens if a link fail?

switches update their tables only considering the probes they receive

- Shall we run the probes more or less frequently?
- What's the right flowlet timeout?

About HULA

- What happens if a link fail?

switches update their tables only considering the probes they receive

- Shall we run the probes more or less frequently?

frequent probes generate more overhead but provide better network visibility

- What's the right flowlet timeout?

About HULA

- What happens if a link fail?

switches update their tables only considering the probes they receive

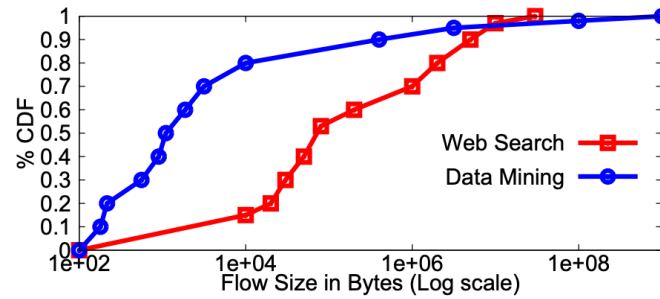
- Shall we run the probes more or less frequently?

frequent probes generate more overhead but provide better network visibility

- What's the right flowlet timeout?

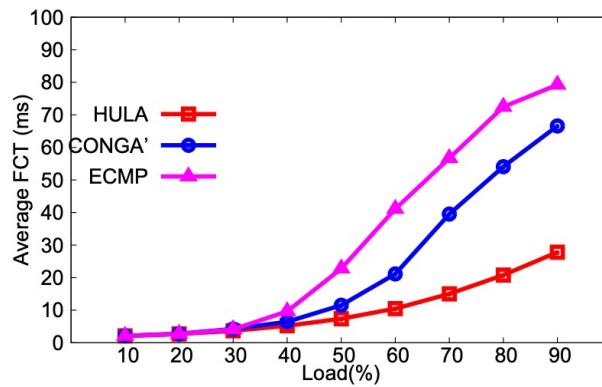
It depends on the network RTT – round trip time

Does it work?

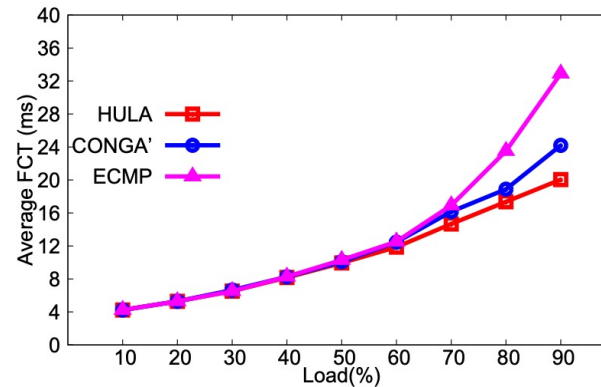


Workloads

3-tier
Fat-Tree

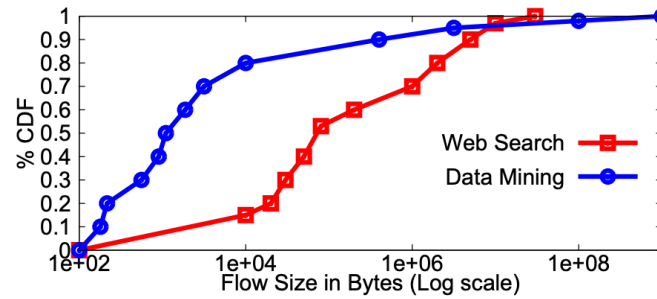


web search



data mining

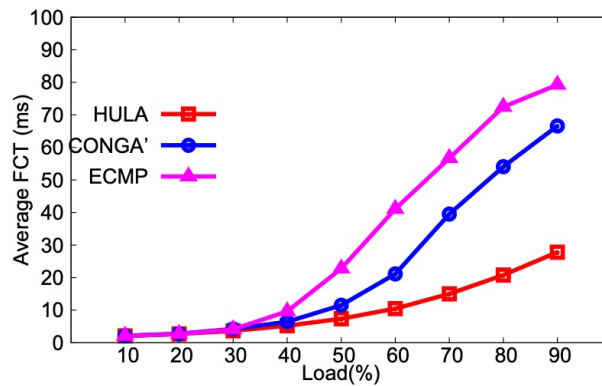
Does it work?



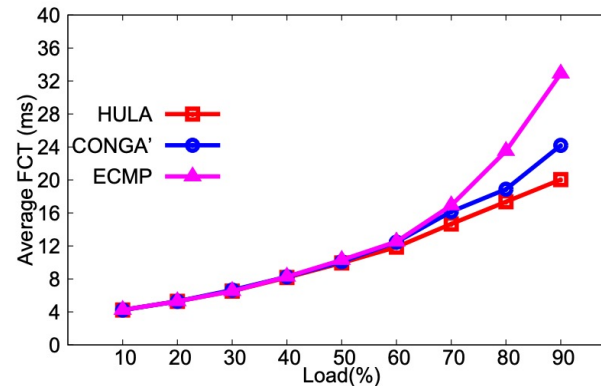
Workloads

3-tier
Fat-Tree

Data mining is mostly composed by short flows.
ECMP is good with them!



web search



data mining